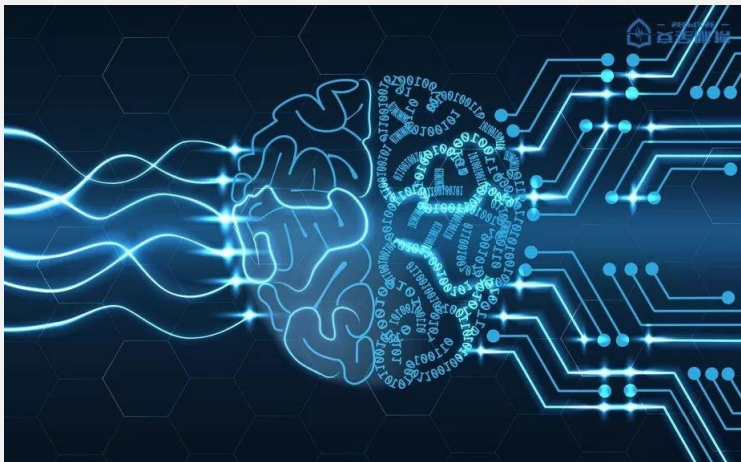




机器学习

Machine Learning

授课老师：程骋

办公室：南一楼中315

邮箱：c_cheng@hust.edu.cn

第九章、强化学习

目录 CONTENTS

01 引言与基本术语

02 马尔科夫决策过程描述强化学习任务

03 K-摇臂赌博机
实现最大奖赏的探索与利用机制及算法

04 有模型学习
环境参数已知的情況

05 基于价值的强化学习
Deep Q learning

06 基于策略的强化学习

为人师表

1: 强化学习是走向AGI的途径之一



AlphaGo: 2016年
战胜李世石, 2017
年战胜柯洁。

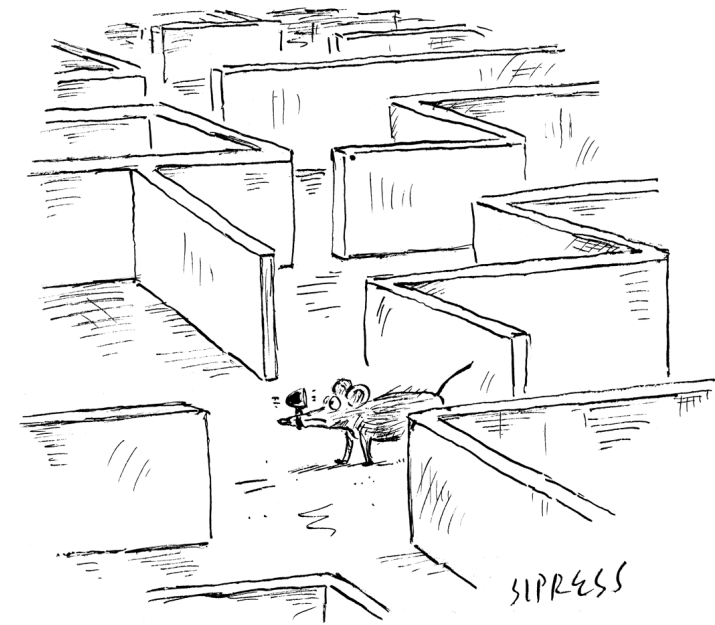
1.0版: 人类棋局

2.0版: 自我对弈产
生棋局

核心: 强化学习与
蒙特卡洛搜索



黄士杰



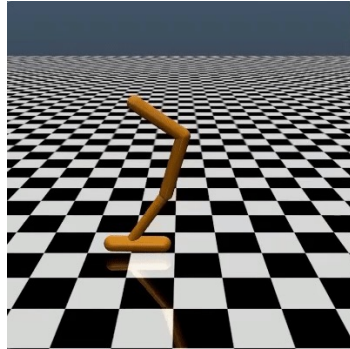
"Recalculating ... recalculating ..."

强化学习 (reinforcement learning) 被认为是人类通往通用人工智能 (AGI) 的有效途径。从以Deepmind为代表的研发团队对强化学习在游戏博弈中的突出表现来看, 强化学习的无监督的学习方法所展现的效果惊人。

1: 强化学习应用的例子



机器人取物



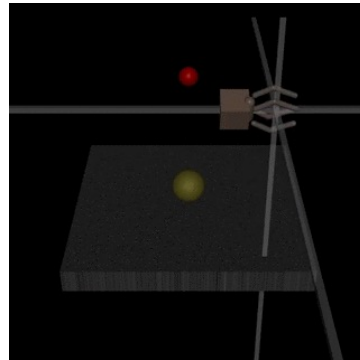
模仿人翻滚



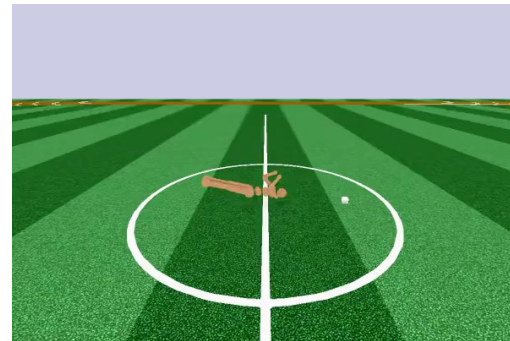
Atari游戏



机器人取物



取物



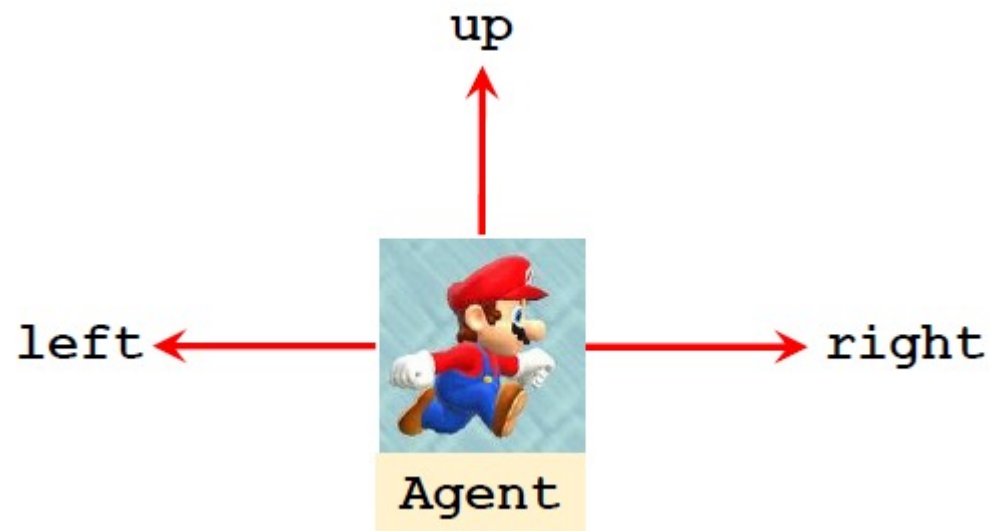
教机器人躲避

1.2、强化学习术语

➤ 状态 (state) 和 行动 (action)



状态：智能体从环境中获取的信息



行动：智能体的行为表征

1.2、强化学习术语

➤ 奖励 (reward)



reward R

- 获得一个金币: $R = +1$
- 游戏获胜: $R = +10000$
- 触碰到蘑菇: $R = -10000$
(game over)
- 无事发生: $R = 0$

1.2、强化学习术语

➤ 状态转移

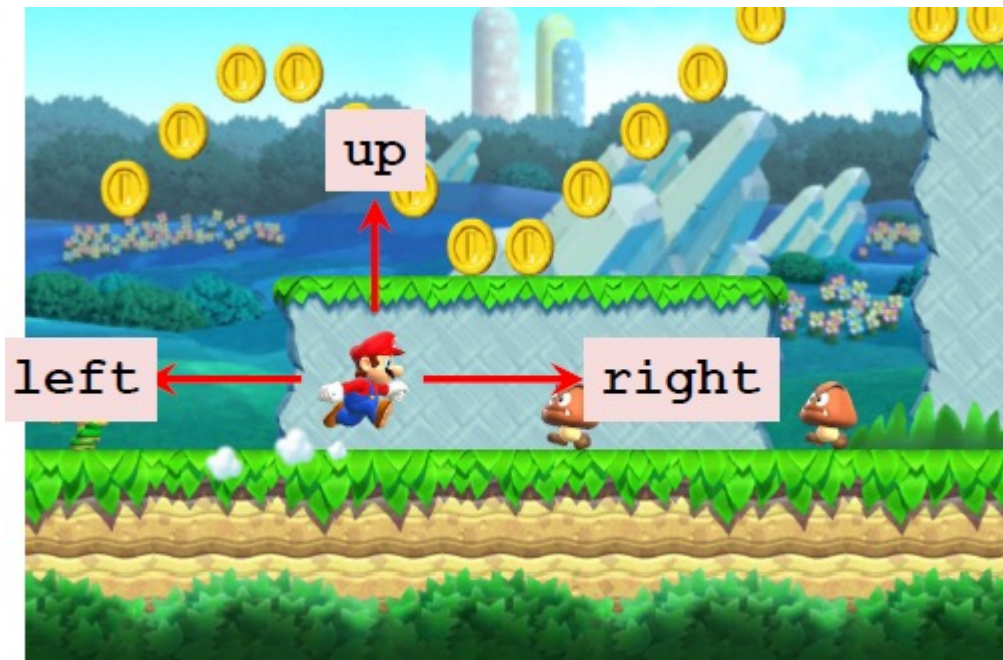


- 例如, “up” 行动会使马里奥进入一个新的状态
- 状态转移可以是**随机**的
- 随机性来源于**环境**影响
- $p(s'|s, a) = P(S' = s' | S = s, A = a)$

1.2、强化学习术语



➤策略 (policy)



策略：智能体根据状态进行下一步行动的函数。

在每个时间步(timestep), 智能体在当前状态根据观察来确定下一步的行动, 因此状态和行动之间存在映射关系, 这种关系就是**策略**, 用 π 表示:

$$\pi: (s, a) \mapsto [0,1]$$

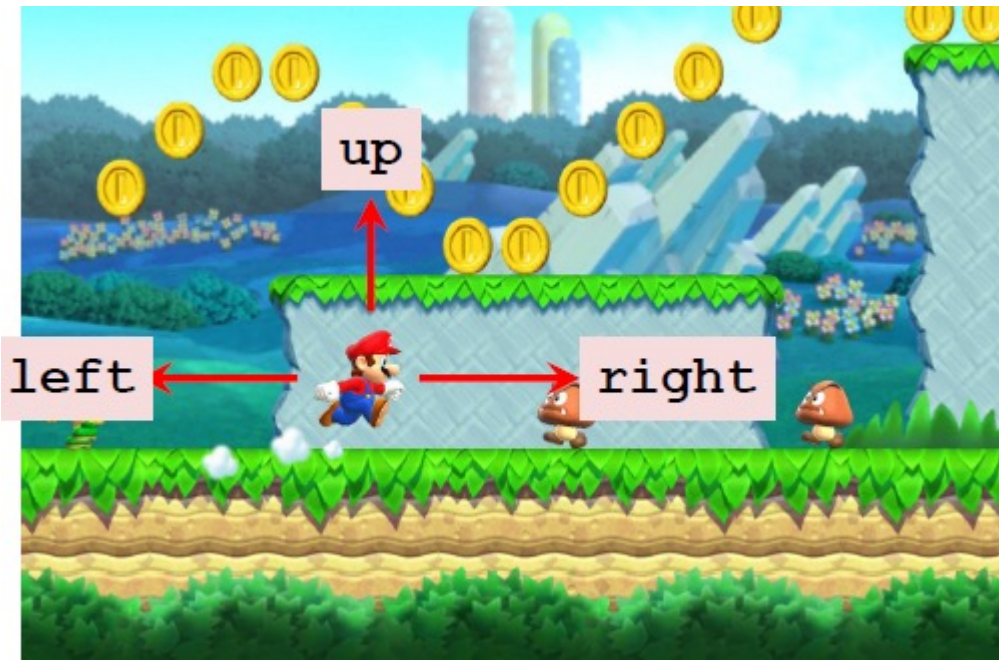
$$\pi(a | s) = P(A = a | S = s)$$

- **确定性策略**: $a = \pi(s)$, 即状态s下执行动作a
- **随机性策略**: $p = \pi(a|s)$, 即在状态s下执行a动作的**概率**

1.2、强化学习术语



➤策略 (policy)

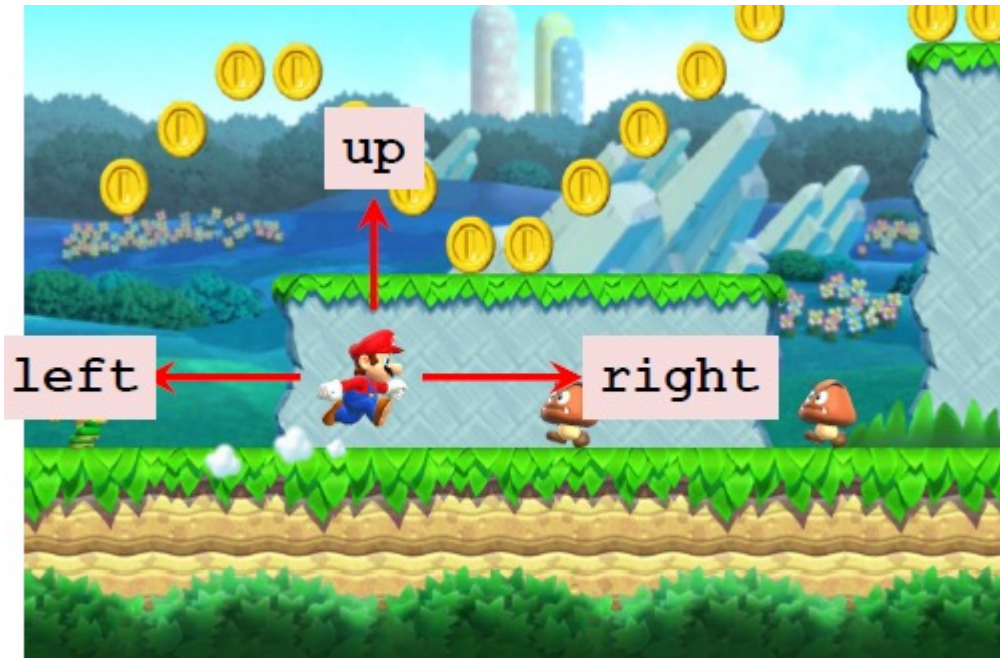


- $\pi(a | s)$ 指当给定状态为 s 时, 采取行动 $A = a$ 的概率
 - $\pi(\text{left} | s) = 0.2$
 - $\pi(\text{right} | s) = 0.1$
 - $\pi(\text{up} | s) = 0.7$
- 观察状态 $S = s$ 时, 智能体行动 A 是**随机**的

1.2、强化学习术语

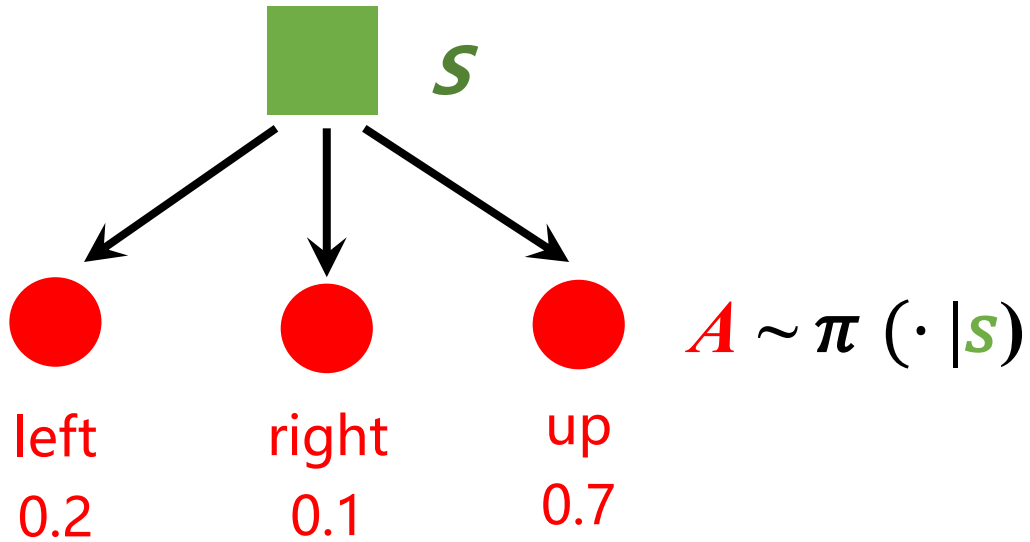
➤策略 (policy)

随机策略还是确定性策略?



1.3、随机性两个来源

➤ 行动随机性

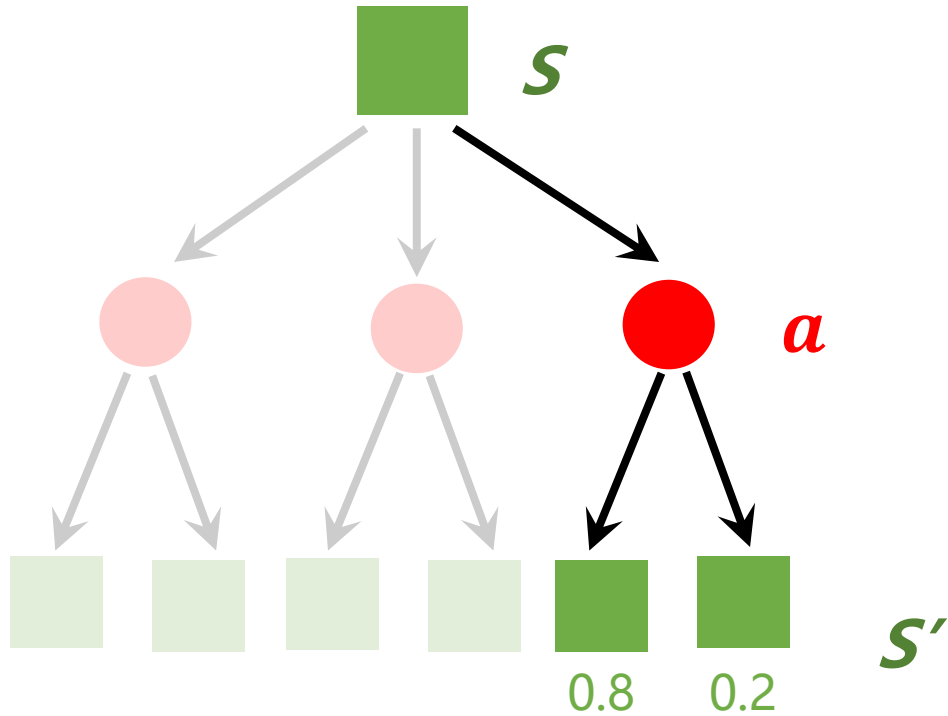


给定状态 s ，行动是随机的，例如：

- $\pi(\text{“left”} | s) = 0.2,$
- $\pi(\text{“right”} | s) = 0.1,$
- $\pi(\text{“up”} | s) = 0.7.$

1.3、随机性两个来源

➤ 状态随机性



- 状态转移是随机的。
- 环境以下面的方式产生新的状态 s'

$$s' \sim p(\cdot | s, a)$$

1.3、随机性两个来源

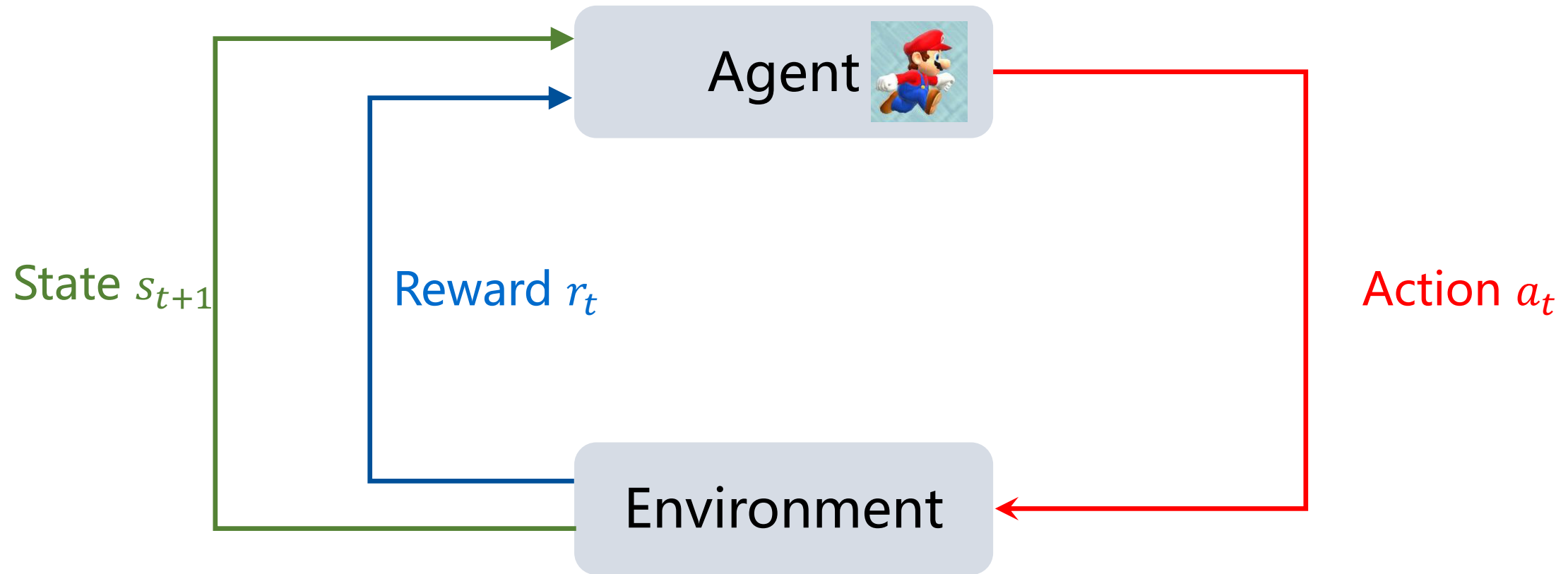
- 行动随机性来源于策略函数：

$$\textcolor{red}{A} \sim \pi(\cdot | \textcolor{green}{s}),$$

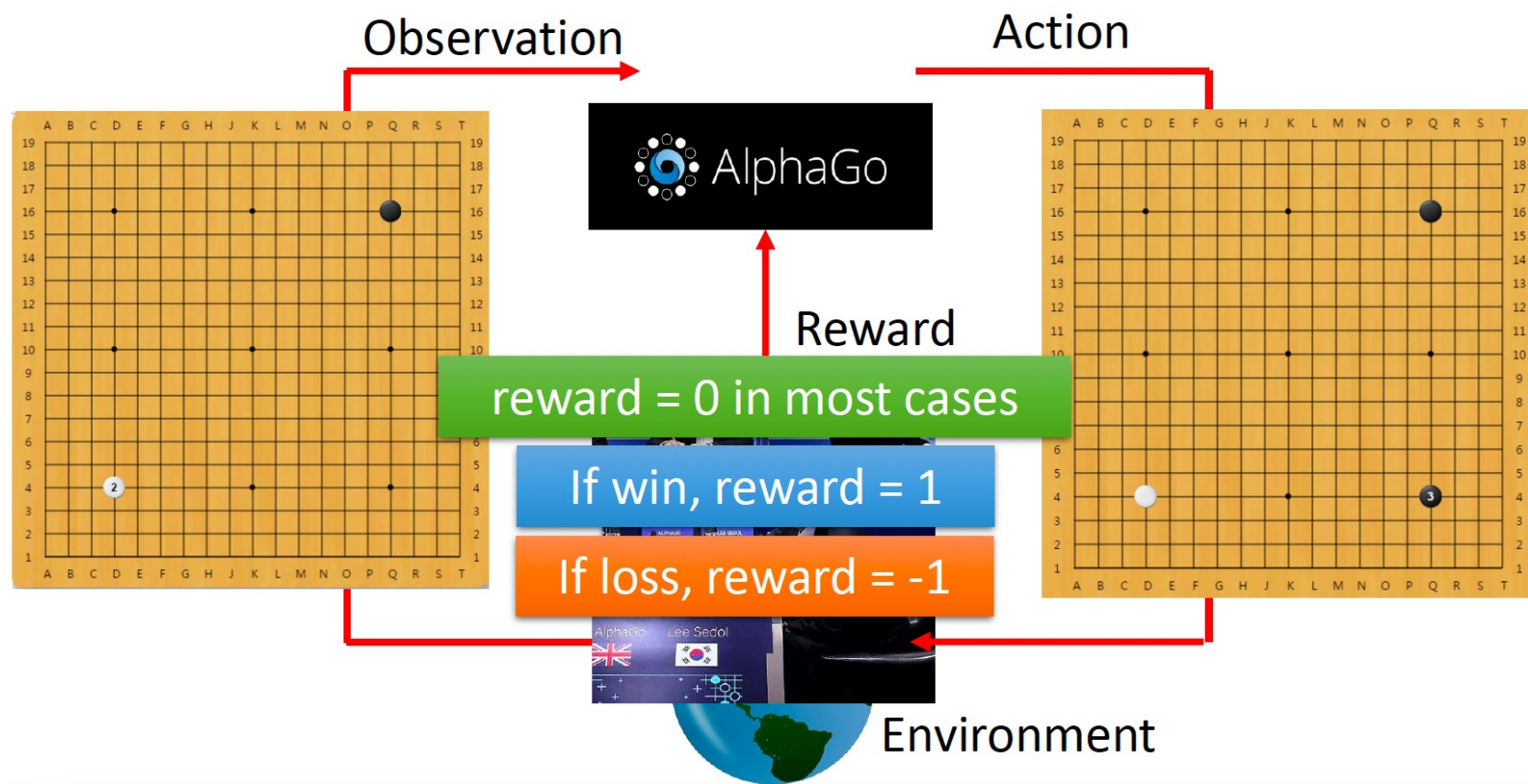
- 状态随机性来源于状态转移函数：

$$\textcolor{green}{s}' \sim p(\cdot | \textcolor{green}{s}, \textcolor{red}{a})$$

1.4、智能体环境交互



1.4、智能体环境交互



1.4 强化学习的典型特征



□ 与其他机器学习方法的不同点

- 仅有奖励信号，没有监督
- 非瞬时的延迟反馈
- 数据并非iid
- 智能体的行动影响其后续接收到的数据

- ◆ 训练数据和测试数据混合使用
- ◆ 不断与环境交互，通过试错，完成策略评估

1.4、智能体环境交互



➤ 运用AI玩游戏

- 观察状态 s_t , 选择行动 $a_t \sim \pi(\cdot | s_t)$, 并执行 a_t 。
- 环境给出新状态 s_{t+1} 和奖励 r_t



1.4、智能体环境交互

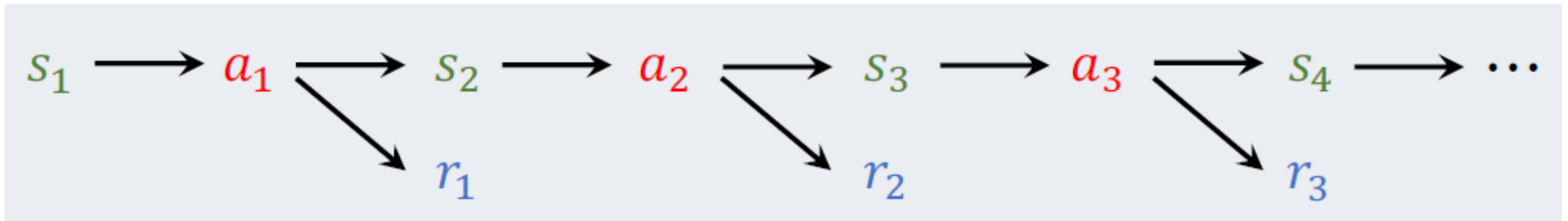


➤ 运用AI玩游戏

- (状态, 行动, 奖励) 轨迹

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_n, a_n, r_n.$$

- 一局从开始到结束 (Mario wins or dies)



1.4、智能体环境交互



➤ 运用AI玩游戏

- (状态, 行动, 奖励) 轨迹

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_n, a_n, r_n.$$

- 一局从开始到结束 (Mario wins or dies)
- 什么是好的策略?
- 好的策略会使累积奖励值增大: $\sum_{t=1}^n \gamma^{t-1} \cdot r_t$
- 使用奖励来引导策略学习

1.5、奖励（奖赏）和回报

➤ 回报(return)

定义： **回报**即**未来的累积奖励**

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

问题：在时刻 t , R_t 和 R_{t+1} 同等重要吗？

- 你喜欢下面哪个？
 - 我现在给你~~\$100~~，\$50
 - 一年后我会给你\$100。
- 未来的奖励不如现在的有价值
- R_{t+1} 的权重应该比 R_t 小。

1.5、奖励和回报

➤ 回报(return)

定义：回报即未来的累积奖励

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

定义：折扣回报(discounted return, 又名未来的累积折扣奖励)

- γ : 折扣因子 (超参数)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

1.5、奖励和回报

➤ 回报(return)

定义：折扣回报（在时刻 t ）

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

在游戏结束，观测 u_t

- 观察所有的奖励， $r_t, r_{t+1}, r_{t+2}, \dots, r_n$
- 可以计算折扣回报：

$$u_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{n-t} r_n.$$

1.5、奖励和回报

➤ 回报(return)

定义：折扣回报（在时刻 t ）

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

在时刻 t ，奖励 $R_t, \dots, R_n$ 是随机的，因此回报 U_t 是随机的。

- 奖励 R_i 依赖于 S_i 和 A_i 。
- 状态可以是随机的: $S_i \sim p(\cdot | s_{i-1}, a_{i-1})$
- 行动可以是随机的: $A_i \sim \pi(\cdot | s_i)$
- 如果 S_i 或 A_i 是随机的，那么 R_i 也是随机的。

1.5、奖励和回报

➤ 回报(return)

定义：折扣回报（在时刻 t ）

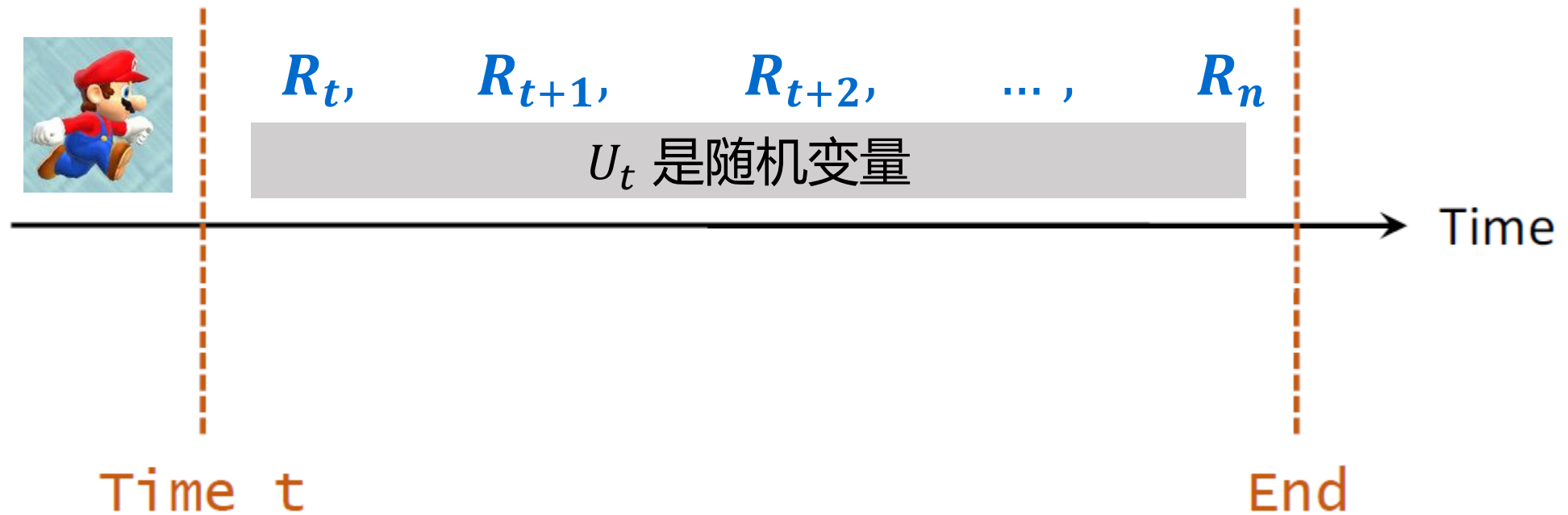
- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

在时刻 t , 奖励 $R_t, \dots, R_n$ 是随机的, 因此回报 U_t 是随机的。

- 奖励 R_i 依赖于 S_i 和 A_i 。
- U_t 依赖于 $R_t, R_{t+1}, \dots, R_n$
- $\rightarrow U_t$ 依赖于 $S_t, A_t, S_{t+1}, A_{t+1}, \dots, S_n, A_n$ 。

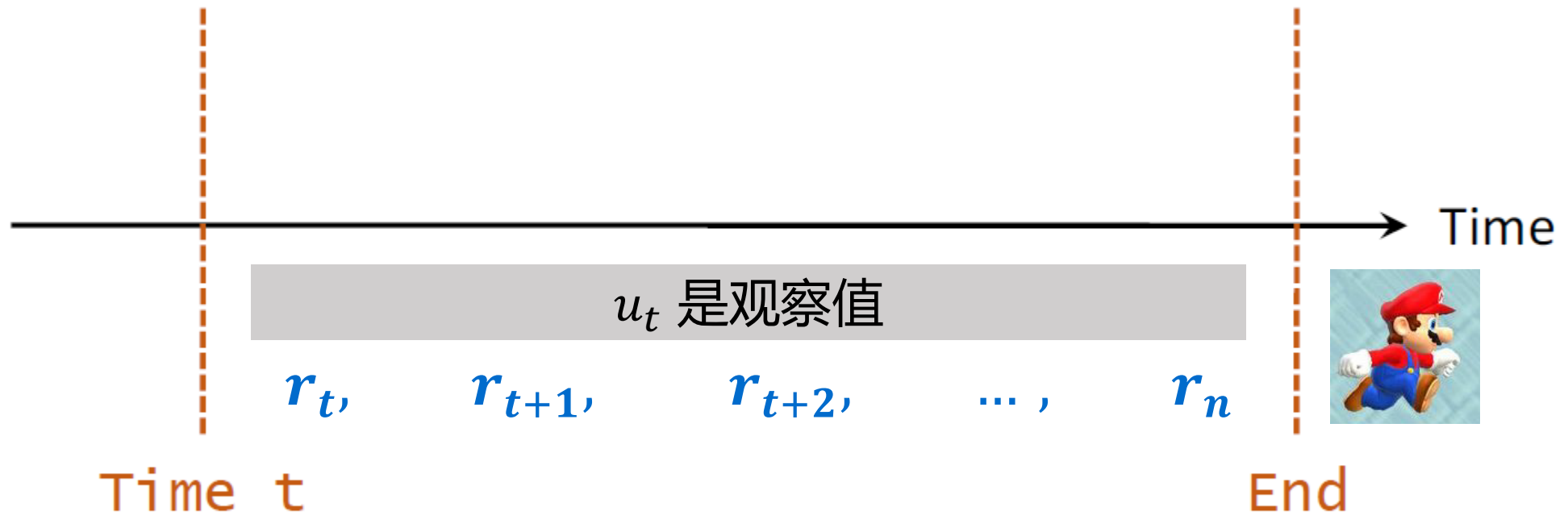
1.5、奖励和回报

➤ 回报随机性



1.5、奖励和回报

➤ 回报随机性



1.6、价值函数(Value Function)

➤ 行动价值函数 $Q_{\pi}(s, a)$

定义：折扣回报（在时刻 t ）

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

定义：行动价值函数（Action-value function）

- $Q_{\pi}(s_t, a_t) = E [U_t | S_t = s_t, A_t = a_t]$

1.6、价值函数(Value Function)

➤ 行动价值函数 $Q_{\pi}(s, a)$

定义：折扣回报（在时刻 t ）

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

定义：行动价值函数（Action-value function）

- $Q_{\pi}(s_t, a_t) = E [U_t | S_t = s_t, A_t = a_t]$

- $Q_{\pi}(s_t, a_t)$ 取决于 s_t, a_t, π, p

- $Q_{\pi}(s_t, a_t)$ 取决于 $s_{t+1}, \dots, s_n$ 和 $a_{t+1}, \dots, a_n$

1.6、价值函数(Value Function)

➤ 状态价值函数 $V_\pi(s)$

定义：折扣回报 (在时刻 t)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

定义：行动价值函数 (Action-value function)

- $Q_\pi(s_t, a_t) = E[U_t | S_t = s_t, A_t = a_t]$

定义：状态价值函数 (State-value function)

- $V_\pi(s_t) = E_A[Q_\pi(s_t, A)] = \sum_a \pi(a | s_t) \cdot Q_\pi(s_t, a)$ (行动是离散的)

$A \sim \pi(\cdot | s_t)$

1.6、价值函数(Value Function)



➤ 状态价值函数 $V_\pi(s)$

定义：折扣回报（在时刻 t ）

$$U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$$

定义：行动价值函数（Action-value function）

$$Q_\pi(s_t, a_t) = E[U_t | S_t = s_t, A_t = a_t]$$

定义：状态价值函数（State-value function）

$$V_\pi(s_t) = E_A[Q_\pi(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_\pi(s_t, a) \quad (\text{行动是离散的})$$

$$V_\pi(s_t) = E_A[Q_\pi(s_t, A)] = \int \pi(a|s_t) \cdot Q_\pi(s_t, a) da \quad (\text{行动是连续的})$$

1.6、价值函数(Value Function)

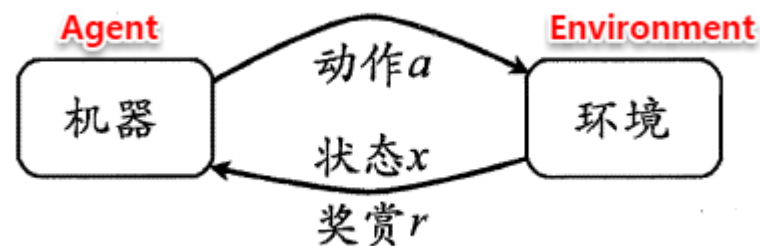


- 行动价值函数: $Q_{\pi}(s, a) = E[U_t | S_t = s, A_t = a]$
- 给定策略 π , $Q_{\pi}(s, a)$ 可以用来评估智能体在状态 s 时选择行动 a 有多好
- 状态价值函数: $V_{\pi}(s) = E_A[Q_{\pi}(s, A)]$
- 给定策略 π , $V_{\pi}(s)$ 评估了智能体在状态 s 时状态有多好

2. 马尔科夫决策过程描述强化学习任务

□ 种西瓜实例

- 种西瓜**任务**：在合适的阶段（**状态**）选种-浇水-施肥-除草-杀虫（**动作**）一系列耕作过程，目标是能种出好西瓜
- 选种-浇水-施肥-除草-杀虫程序做得好不好，只有西瓜成熟后才知道是否获得**回报**
- 多次种瓜可以摸索出好的种瓜**策略**（如何实施以上动作），这个过程进行抽象即为**强化学习**

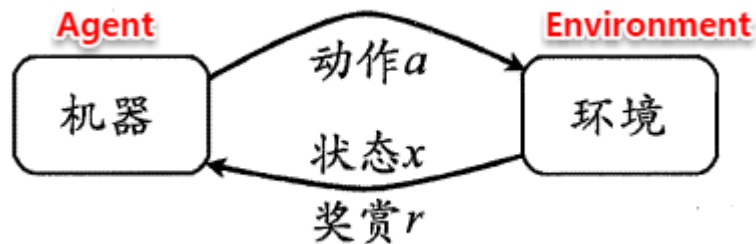


强化学习图示

2: 马尔科夫决策过程描述强化学习任务



- 在一个环境E中，每个状态为机器对当前环境的感知；机器只能通过动作来影响环境，当机器执行一个动作后，会使得环境按某种概率转移到另一个状态；同时，环境会根据潜在的奖赏函数反馈给机器一个奖赏



四元组定义强化学习任务

$$E = \langle X, A, P, R \rangle$$

状态转移概率P: $X \times A \times X \rightarrow \mathcal{R}$

奖赏R: $X \times A \times X \rightarrow \mathcal{R}$

状态X

对环境的感知，所有可能的状态称为状态空间，例如种西瓜任务中西瓜长势的描述

动作A

机器所采取的动作，所有能采取的动作构成动作空间。例如浇水、施肥等可选的动作

转移概率P

当执行某个动作后，当前状态会以某种概率转移到另一个状态。例如西瓜缺水，在浇水动作后，瓜苗长势发生变化

奖赏函数R

当在状态转移的同时，环境反馈给机器一个奖赏。例如瓜苗健康+1,瓜苗凋零-10

2: 马尔科夫决策过程描述强化学习任务

□ 西瓜浇水为例

- 四个状态：健康、缺水、溢水、凋亡
- 两个动作：浇水、不浇水
- 转移：采取动作后，状态变化的概率
- 奖赏：保持健康+1，缺水或溢水-1，凋亡-100

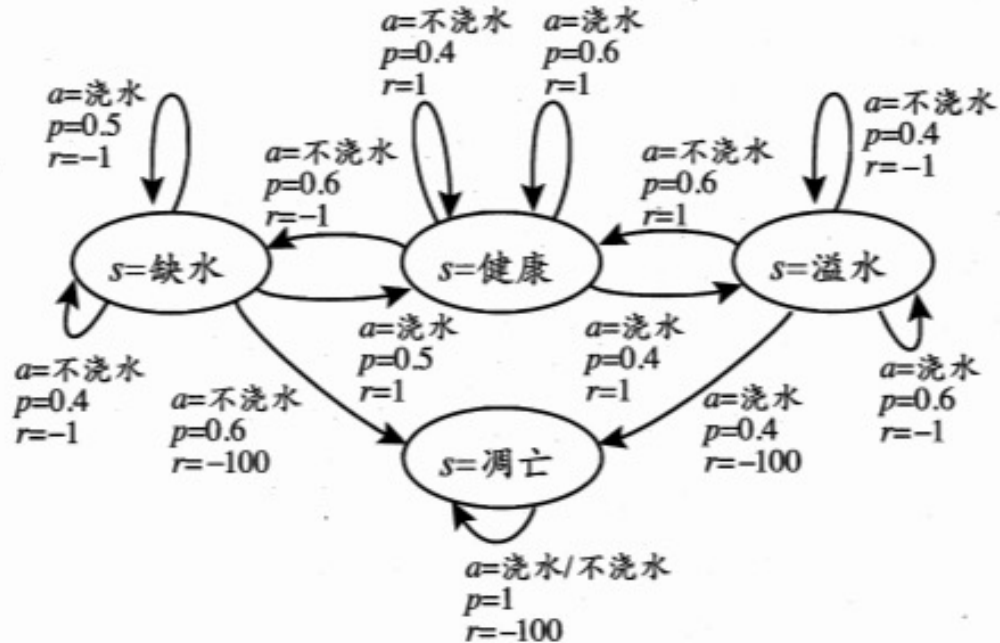


图 16.2 给西瓜浇水问题的马尔可夫决策过程

- 箭头表示状态转移
- a, p, r 分别表示导致状态转移的动作、转移概率以及奖赏
- **最优策略**：健康-浇水；溢水-不浇水；缺水-浇水；凋亡-任意动作。

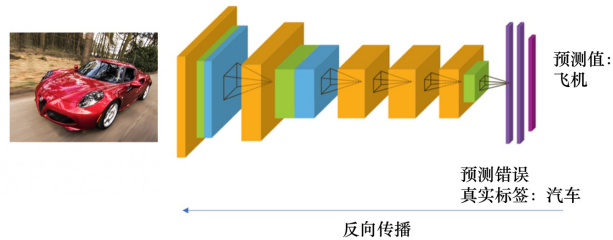
2: 马尔科夫决策过程描述强化学习任务



- 通过在环境中不断地尝试，根据尝试获得的反馈信息调整策略，最终生成一个较好的策略 π ，机器根据这个策略便能知道在什么状态下执行什么动作
- 策略表示方法常有两种
 - 确定性策略: $a = \pi(x)$ ，即状态 x 下执行动作 a
 - 随机性策略: $p = \pi(x, a)$ ，即在状态 x 下执行 a 动作的概率
- 一个策略的优劣取决于长期执行这一策略后的累积奖赏，即使用累积奖赏来评估策略的好坏，最优策略则表示在初始状态下一直执行该策略后，最后的累积奖赏值最高。某策略使瓜苗枯死，则累积值低；种出好瓜的策略，则值高。
- 学习目的：找到长期累积奖赏最大化的策略。长期累积的计算公式常用两种：

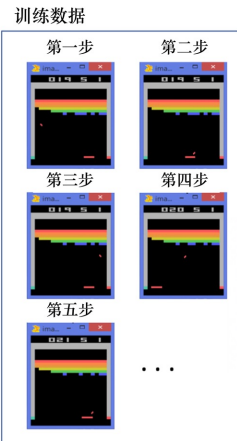
- T步累积奖赏: $\mathbb{E}[\frac{1}{T} \sum_{t=1}^T r_t]$ ，即执行该策略T步的平均奖赏的期望值。
- γ 折扣累积奖赏: $\mathbb{E}[\sum_{t=0}^{+\infty} \gamma^t r_{t+1}]$ ，一直执行到最后，同时越往后的奖赏权重越低。

2: 强化学习与监督学习的关系

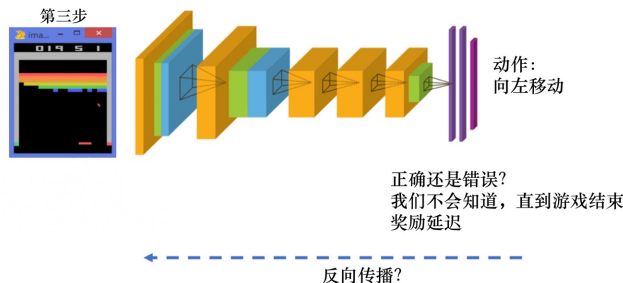


□ 监督学习训练过程 不同: 采样分布要求; 标注要求; 目标;

- 输入数据 x_i , 其服从iid数据分布
- 使用模型 (参数为 w) 预测输出 y
- 观测目标输出 y_i 和损失 $l(w, x_i, y_i)$
- 更新 w 以降低损失 $w \leftarrow w - \eta \nabla l(w, x_i, y_i)$



人类在玩?



□ 强化学习训练过程

分类器 策略 示例 标记 状态 动作

- 从状态 x , 采取由策略 $\pi(x)$ 确定的行动 a
- 环境基于转移模型 $P(x'|x, a)$ 选择下一状态 x'
- 观测状态 x' 和奖励 $r(x')$
- 更新策略



- 前后两帧有时间相关性, 不服从iid
- 没有强烈的监督者, 只有奖励信号
- 奖励是延迟的, 如游戏结束才知道动作好坏
- 通过不断地试错探索才能获知选出最佳动作的能力

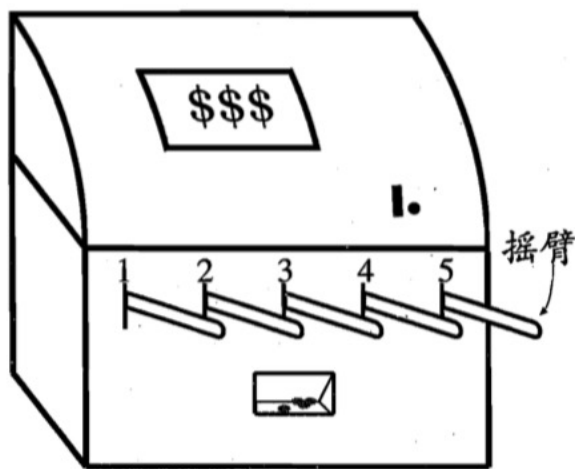
□ 强化学习是 “延迟标记信息” 的监督学习问题

3.K摇臂赌博机



- 强化学习的最终奖赏需多步动作后才知道
- 简化：最大化单步奖赏。在状态 x 下执行一次动作 a 便能观察到奖赏结果
- 问题：各个动作产生的结果需要尝试才知道，无训练数据提前告知

- 若每个动作的奖赏为确定值，则每个动作尝试一遍即可
- 但一般而言，动作的奖赏值来自一个概率分布，试一次得不到平均值



□ 单步强化学习任务对应了一个理论模型K摇臂赌博机

K-摇臂赌博机有 K 个摇臂，赌徒在投入一个硬币后可选择按下其中一个摇臂，每个摇臂以一定的概率吐出硬币，但这个概率赌徒并不知道。赌徒的目标是通过一定的策略最大化自己的奖赏，即获得最多的硬币。

3.1: 探索和利用

- 强化学习是试错学习
- 要找到最佳的策略: 根据对环境经验, 且不损失太多奖赏



探索



利用

- | | |
|---|--|
| <ul style="list-style-type: none"> ● 搜索更多的关于环境的信息 <div style="border: 1px solid red; padding: 5px; margin-top: 10px;"> <ul style="list-style-type: none"> ● 试新的餐馆 ● 石油勘探试新的位置 ● 玩游戏时选取新的策略 </div> | <ul style="list-style-type: none"> ● 利用已知信息最大化奖赏 <div style="border: 1px solid red; padding: 5px; margin-top: 10px;"> <ul style="list-style-type: none"> ● 去最喜爱的餐馆 ● 石油勘探在已知最佳的位置钻孔 ● 玩游戏时选取熟悉的策略 </div> |
|---|--|

3.1：探索和利用



□ 单步强化学习实质上是K-摇臂赌博机的原型，一般我们尝试动作的次数是有限的，那如何利用有限的次数进行有效地探索呢？这里有两种基本的想法：

- **仅探索法**：将所有的尝试机会平均分配给每个摇臂(即轮流按下每个摇臂)，即轮流执行，最后以每个摇臂各自的平均吐币概率作为其奖赏期望的近似估计
 - 很好地估计每个摇臂的奖赏，却会失去很多选择最优摇臂的机会
- **仅利用法**：按下目前最优的(即到目前为止平均奖赏最大的)摇臂，若有多
个摇臂同为最优，则从中随机选取一个
 - 没有很好地估计摇臂期望奖赏，很可能经常选不到最优摇臂

结论：探索(即估计摇臂的优劣)和利用(即选择当前最优摇臂)这两者是矛盾的，因为尝试次数(即总投币数)有限，加强了一方则会自然削弱另一方，这就是强化学习所面临的**探索-利用窘境**(Exploration-Exploitation dilemma)。想要累积奖赏最大，则必须在探索与利用之间达成较好的折中。

3.2: ϵ -贪心

□ ϵ -贪心法基于一个概率来对探索和利用进行折中，即每次尝试时：

- **以 ϵ 的概率进行探索**：即以均匀概率随机选择一个动作
- **以 $1-\epsilon$ 的概率进行利用**：即选择当前最优的动作

□ 令 $Q(k)$ 为摇臂 k 的平均奖赏， v_i 为第 i 次的奖赏，则平均奖赏：

$$Q(k) = \frac{1}{n} \sum_{i=1}^n v_i$$

□ 均值的**增量式公式**：

$$\begin{aligned} Q_n(k) &= \frac{1}{n} ((n-1)Q_{n-1}(k) + v_n) \\ &= Q_{n-1}(k) + \frac{1}{n} (v_n - Q_{n-1}(k)) \end{aligned}$$

输入: 摇臂数 K ;

奖赏函数 R ;

尝试次数 T ;

探索概率 ϵ .

过程:

1: $r = 0$;

2: $\forall i = 1, 2, \dots, K : Q(i) = 0$ $\text{count}(i) = 0$

平均奖赏

选中次数

3: **for** $t = 1, 2, \dots, T$ **do**

4: **if** $\text{rand}() < \epsilon$ **then**

5: $k =$ 从 $1, 2, \dots, K$ 中以均匀分布随机选取

6: **else**

7: $k = \arg \max_i Q(i)$

8: **end if**

9: $v = R(k)$;

10: $r = r + v$;

11: $Q(k) = \frac{Q(k) \times \text{count}(k) + v}{\text{count}(k) + 1}$; 增量式更新

12: $\text{count}(k) = \text{count}(k) + 1$;

13: **end for**

输出: 累积奖赏 r

□ 若奖赏不确定性大, 例如概率分布较宽时, 则需更多的探索, 设置较大的 ϵ

□ 若奖赏不确定性小, 例如概率分布较集中时, 则少量的尝试就可很好地近似真实奖赏, 需要较小的 ϵ

□ 通常 ϵ 取0.1或0.01

3.3: Softmax算法



- 基于当前每个动作的平均奖赏值来对探索和利用进行折中，Softmax函数将一组值转化为一组概率，Boltzman分布：

$$P(k) = \frac{e^{\frac{Q(k)}{\tau}}}{\sum_{i=1}^K e^{\frac{Q(i)}{\tau}}},$$

T : 温度
T -> 0: 越放大差距
T -> inf : 越缩小差距

- **各摇臂平均奖赏相当时**：各摇臂选取概率相当。 $\tau \rightarrow \text{inf}$ ，仅探索
- **某些摇臂平均奖赏明显高时**：被选取的概率也明显高。 $\tau \rightarrow 0$ ，仅利用

输入：摇臂数 K ;
 奖赏函数 R ;
 尝试次数 T ;
 温度参数 τ .

过程:

- 1: $r = 0$;
- 2: $\forall i = 1, 2, \dots, K : Q(i) = 0, \text{count}(i) = 0$;
- 3: **for** $t = 1, 2, \dots, T$ **do**
- 4: $k =$ 从 $1, 2, \dots, K$ 中根据式(16.4)随机选取
- 5: $v = R(k)$;
- 6: $r = r + v$;
- 7: $Q(k) = \frac{Q(k) \times \text{count}(k) + v}{\text{count}(k) + 1}$;
- 8: $\text{count}(k) = \text{count}(k) + 1$;
- 9: **end for**

输出：累积奖赏 r

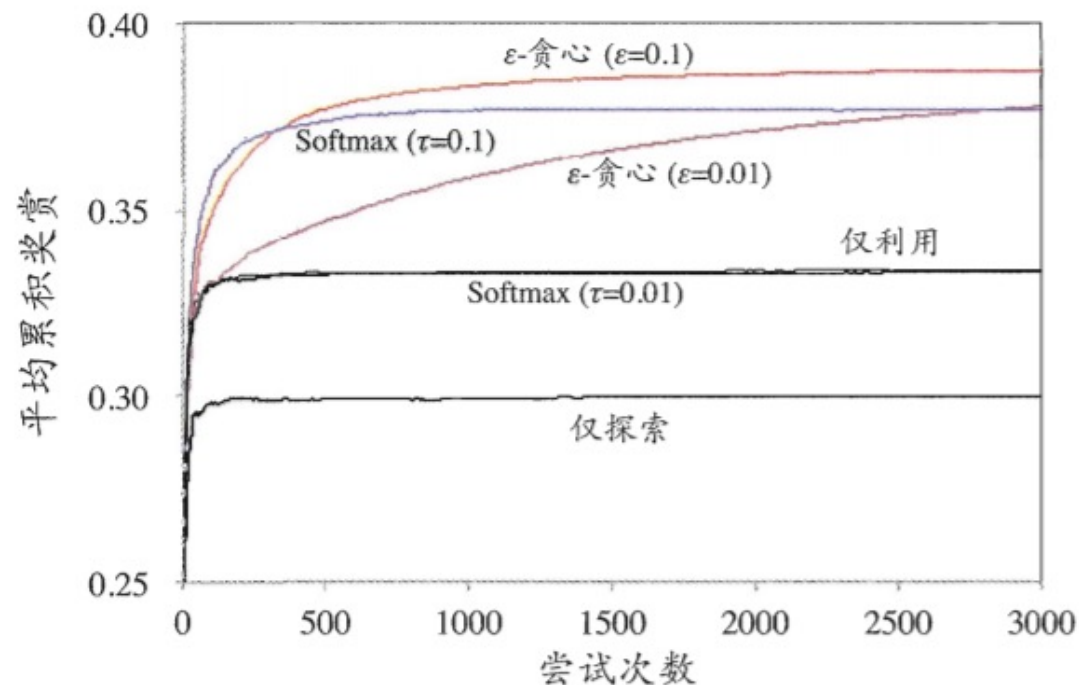
图 16.5 Softmax算法 <https://blog.csdn.net/Tinky2013>

3.4:两种算法的比较



□ ϵ -贪心和Softmax算法的优劣取决于具体应用:

- **例子:** 假定2-摇臂赌博机的摇臂1以0.4的概率返回奖赏1, 以0.6的概率返回奖赏0; 摇臂2以0.2的概率返回奖赏1, 以0.8的概率返回奖赏0



结果: 不同参数下的平均累积奖赏如图所示, 其中每条曲线对应于重复1000次实验的平均结果。Softmax ($\tau=0.01$) 的曲线与“仅利用”曲线几乎重合。

3.5:基于赌博机算法的多步强化学习

□ 对于离散状态空间、离散动作空间上的多步强化学习任务：

- 每个状态上动作的选择视为K-摇臂赌博机问题
- 累积奖赏替代K-摇臂赌博机的单步奖赏函数
- 每个状态分别记录各动作的尝试次数、当前平均累积奖赏函数等信息

执行：基于赌博机算法选择要尝试的动作

□ 但这种方法的缺陷：

- 每个状态相互独立
- 未考虑强化学习任务中状态间的马尔科夫决策过程的特性

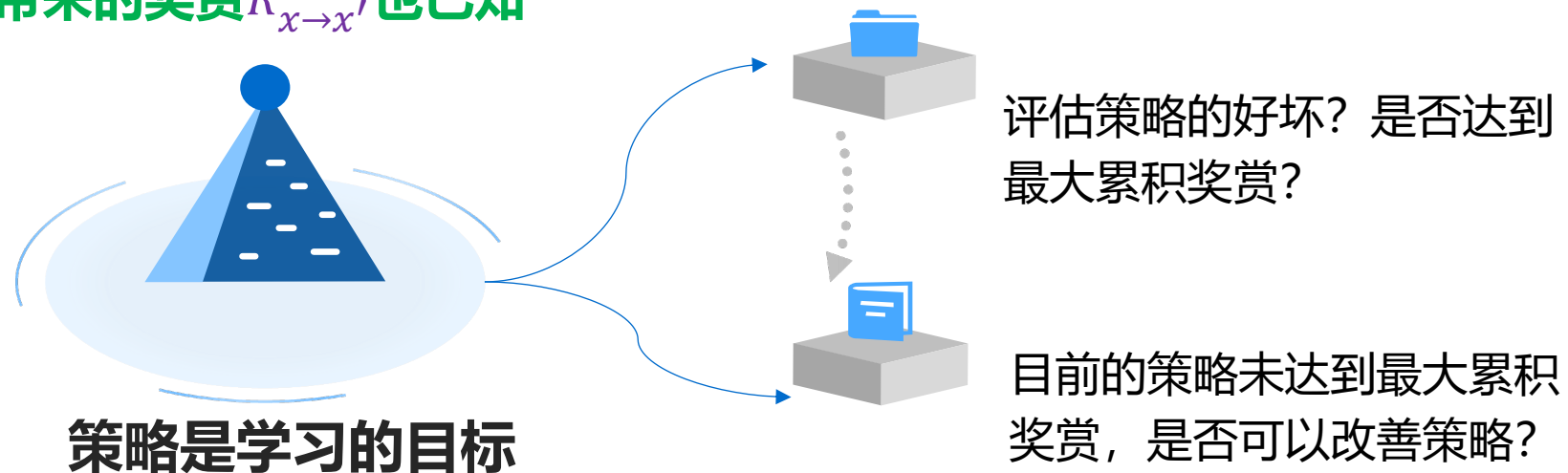
4.有模型学习

□ 模型已知：强化学习任务的马尔科夫决策过程四元组 $E = \langle X, A, P, R \rangle$ 已知

- 机器内部模拟出外部环境相同或近似的状况，可对环境进行建模
- 在已知模型的环境中进行学习称为“有模型学习”

□ 假设状态空间 X 和动作空间均为有限，则在状态 x 执行动作 a :

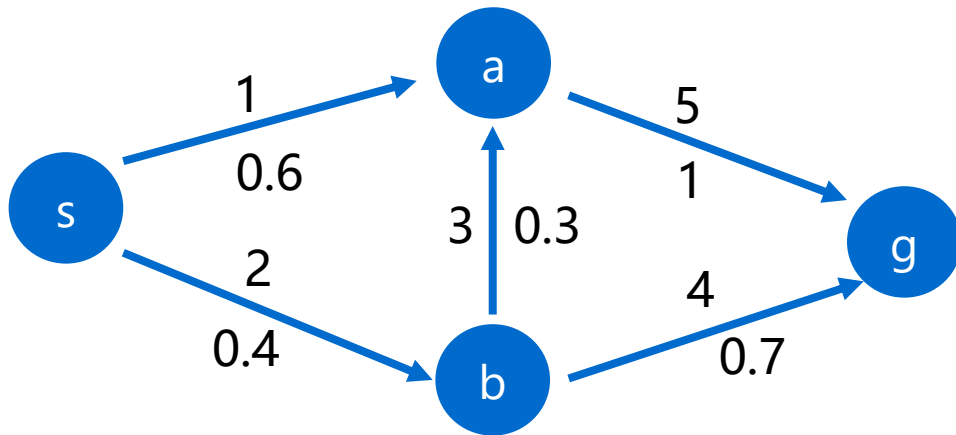
- 转移到状态 x' 的概率 $P_{x \rightarrow x'}^a$ 已知
- 该转移带来的奖赏 $R_{x \rightarrow x'}^a$ 也已知



4. 策略评估

□ 模型已知时，可对任意策略 π 估计出其带来的期望累积奖赏，进而评估策略优劣

- 状态值函数 $V(\cdot)$ ：从状态 x 出发，使用策略 π 获得的累积奖赏 $V^\pi(x)$
- 状态-动作值函数 $Q(\cdot)$ ：从状态 x 出发，执行动作 a 后，再使用策略 π 获得的累积奖赏 $Q^\pi(x, a)$



- 从s到g一共有3种方式,这三种方式:这三个轨迹的回报分别是6、10、6

$$V(s) = 0.6 * 6 + 0.4 * 0.3 * 10 + 0.4 * 0.7 * 6 = 6.48$$

$$v(a) = 5; \quad v(b) = 5.2$$

- 动作价值函数 $Q(s, a1)$ = 选择该动作得到的回报 + 该动作到达的下一个状态的状态价值函数

$$Q(s, a1) = 1 + v(a) = 6 \quad Q(s, a2) = 2 + v(b) = 7.2$$

- 状态价值函数 = 动作1概率 * 动作1的动作价值函数 + 动作2概率 * 动作2的动作价值函数 + 动作i概率 * 动作i的动作价值函数

$$v(s) = 0.6 * 6 + 0.4 * 7.2 = 6.48$$

□ 更多情况下模型未知

5 基于价值的强化学习



➤ 行动价值函数 $Q(s, a)$

$$Q_{\pi}(s_t, a_t) = E[U_t | S_t = s_t, A_t = a_t]$$

定义：最优行动价值函数

$$Q^*(s_t, a_t) = \max_{\pi} Q_{\pi}(s_t, a_t)$$

- 无论采用何种策略 π ，在状态 s_t 下采取行动 a_t 的结果不可能优于 $Q^*(s_t, a_t)$

5.1、Deep Q-Network(DQN)



➤ 近似Q函数

目标：赢得游戏（~最大化总奖励）

Q^* 是用来评判智能体当处于状态 s 时选择行动 a 有多好的指标

问题：若已知 $Q^*(s, a)$ ，应采取的最优行动是什么？

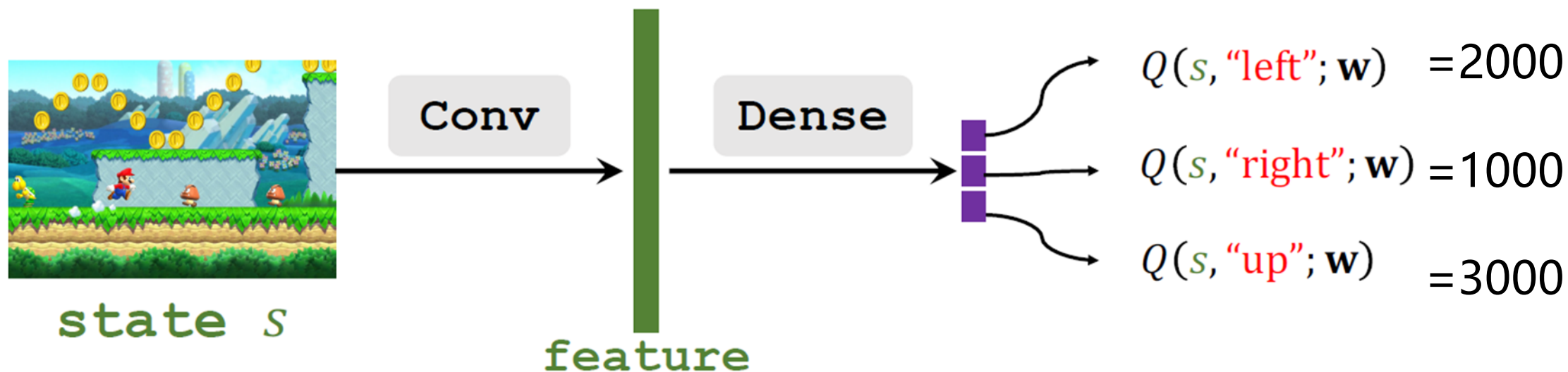
- 显而易见，最优行动是 $a^* = \underset{a}{\operatorname{argmax}} Q^*(s, a)$

挑战： $Q^*(s, a)$ 是未知的

- 解决方案：Deep Q-Network(DQN)
- 使用神经网络 $Q(s, a; \mathbf{w})$ 近似 $Q^*(s, a)$

5.1、Deep Q-Network(DQN)

- Input shape: 屏幕截图大小
- Output shape: 行动空间维度

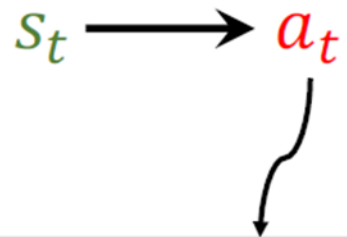


问题：根据预测，应该采取什么行动？

5.1、Deep Q-Network(DQN)



➤应用DQN到打游戏

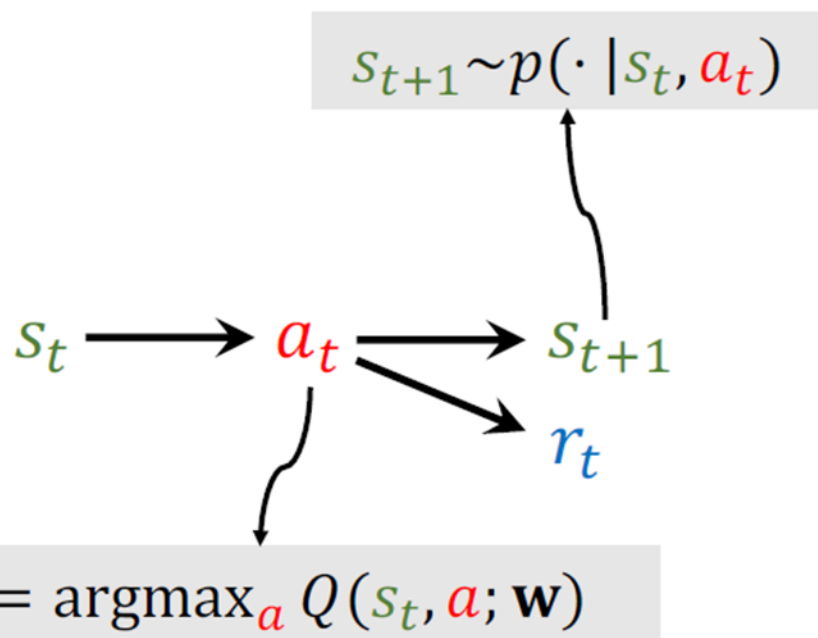


$$a_t = \operatorname{argmax}_a Q(s_t, a; \mathbf{w})$$

5.1、Deep Q-Network(DQN)

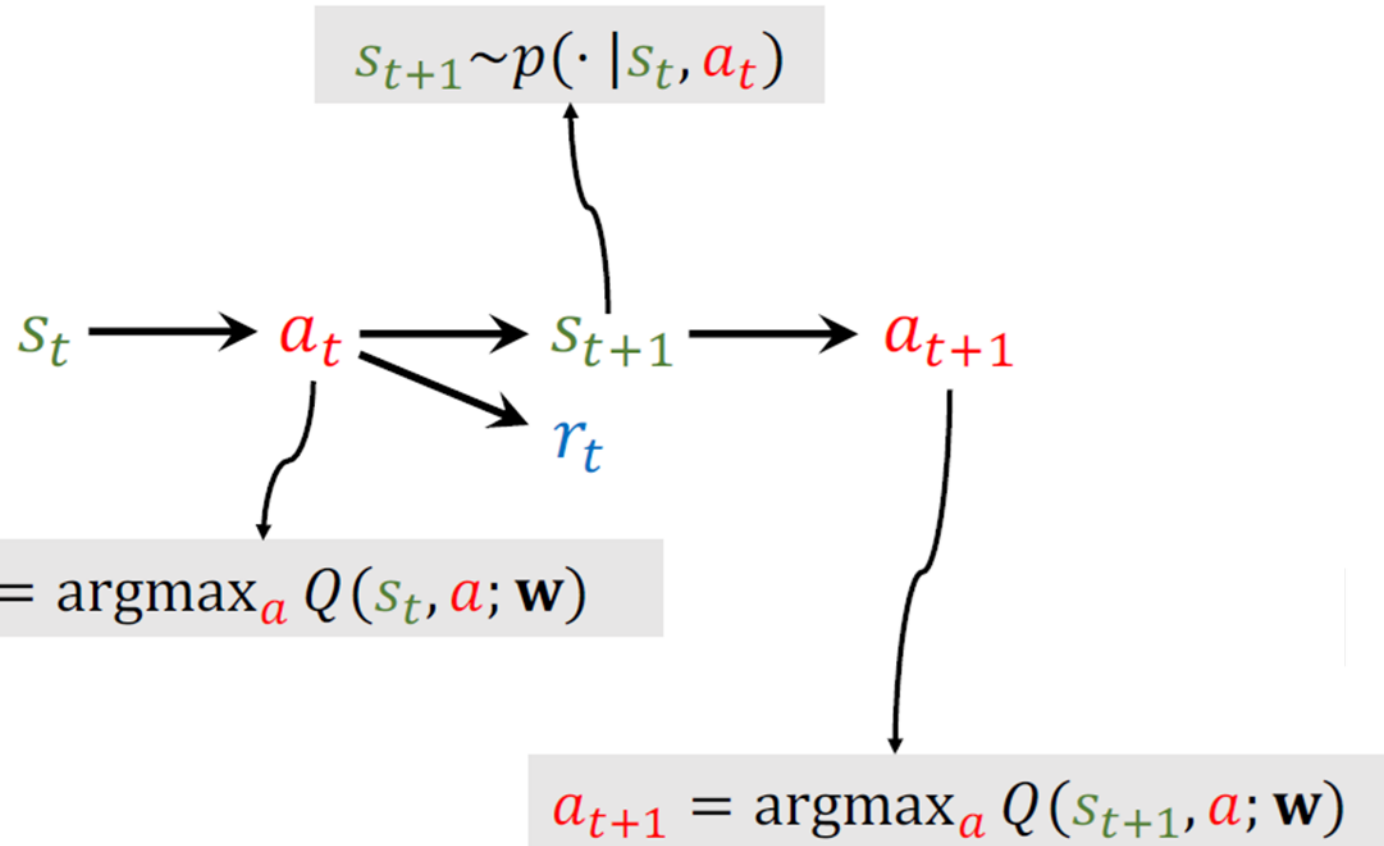


➤应用DQN到打游戏



5.1、Deep Q-Network(DQN)

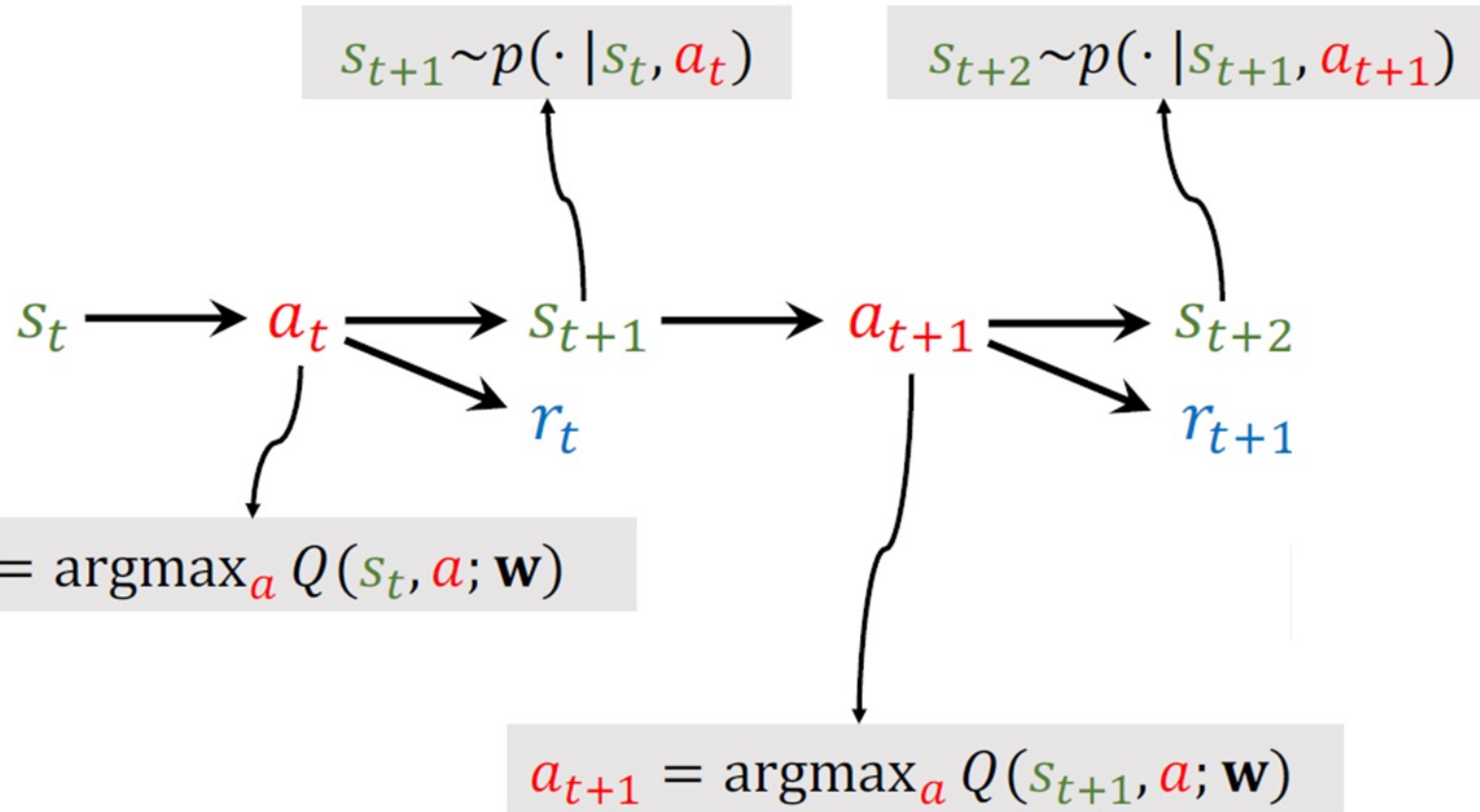
➤应用DQN到打游戏



5.1、Deep Q-Network(DQN)



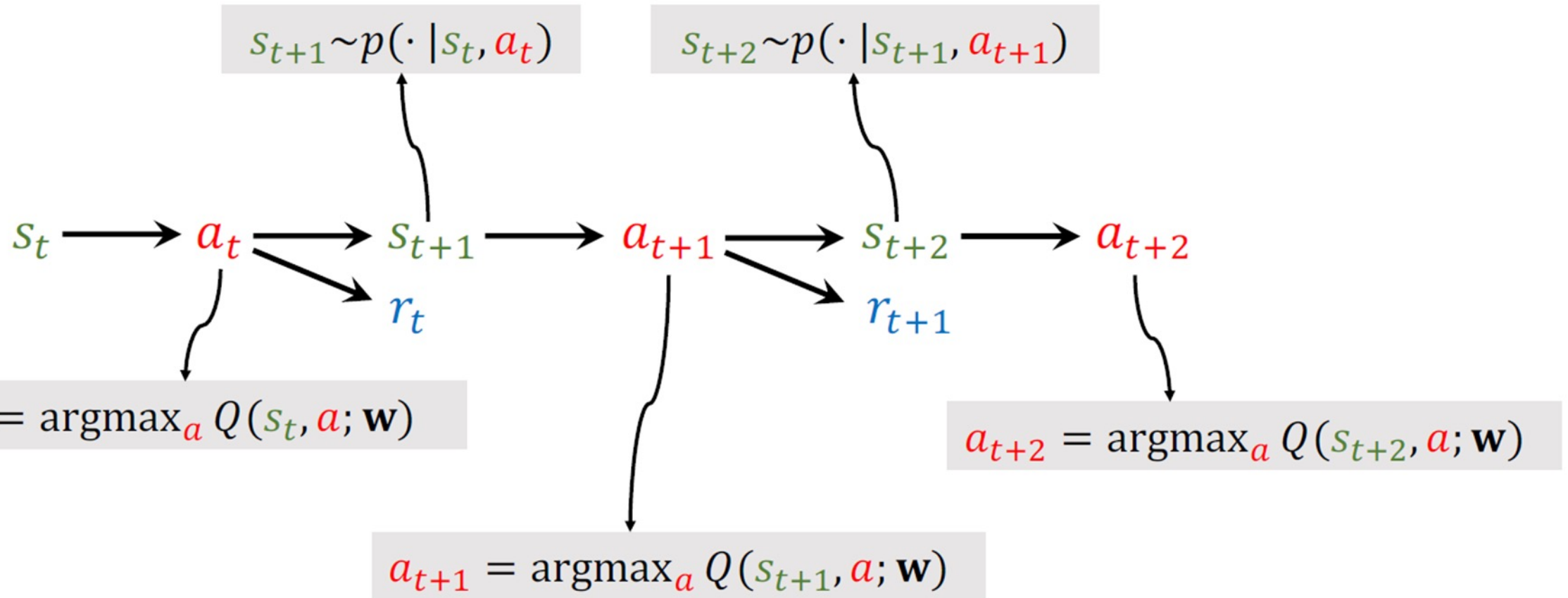
➤应用DQN到打游戏



5.1、Deep Q-Network(DQN)



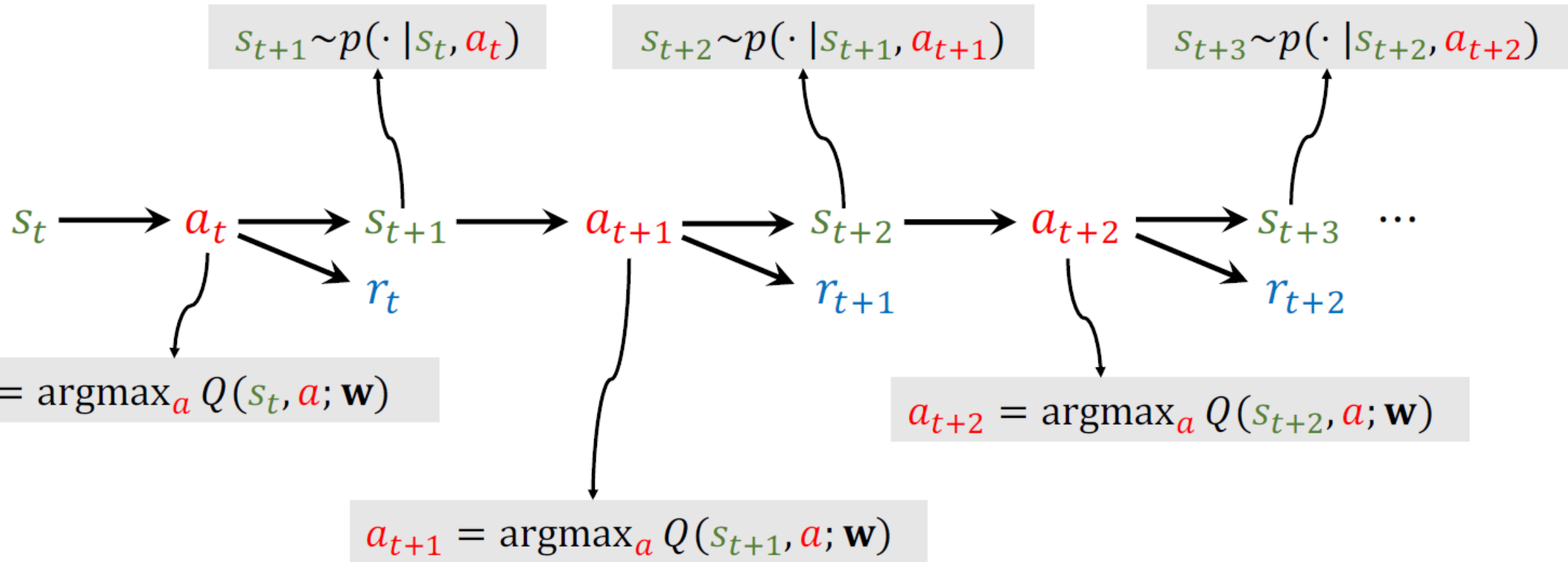
➤应用DQN到打游戏



5.1、Deep Q-Network(DQN)



➤应用DQN到打游戏



5.2、 Temporal Difference (TD) Learning

Reference

1. Sutton and others: A convergent $O(n)$ algorithm for off-policy temporal-difference learning with linear function approximation. In *NIPS*, 2008.
2. Sutton and others: Fast gradient-descent methods for temporal-difference learning with linear function approximation. In *ICML*, 2009.

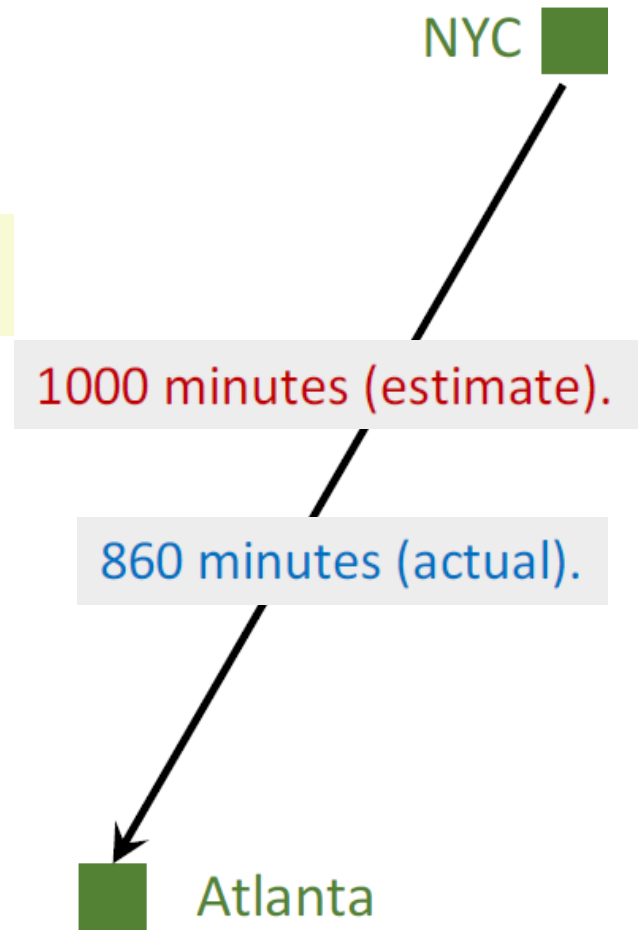
5.2、Temporal Difference (TD) Learning

➤ 举例

- 从纽约开车到亚特兰大需耗时多久？
- 模型 $Q(\mathbf{w})$ 进行时间成本的估计，例如 1000min。

问题：如何更新模型？

- 预测值： $q = Q(\mathbf{w})$ ，例如， $q = 1000$ 。
- 真实值： y ，例如， $y = 860$ 。
- Loss: $L = \frac{1}{2}(q - y)^2$
- 梯度： $\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial q}{\partial \mathbf{w}} \cdot \frac{\partial L}{\partial q} = (q - y) \cdot \frac{\partial Q(\mathbf{w})}{\partial \mathbf{w}}$
- 梯度下降： $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L}{\partial \mathbf{w}} \big|_{\mathbf{w}=\mathbf{w}_t}$



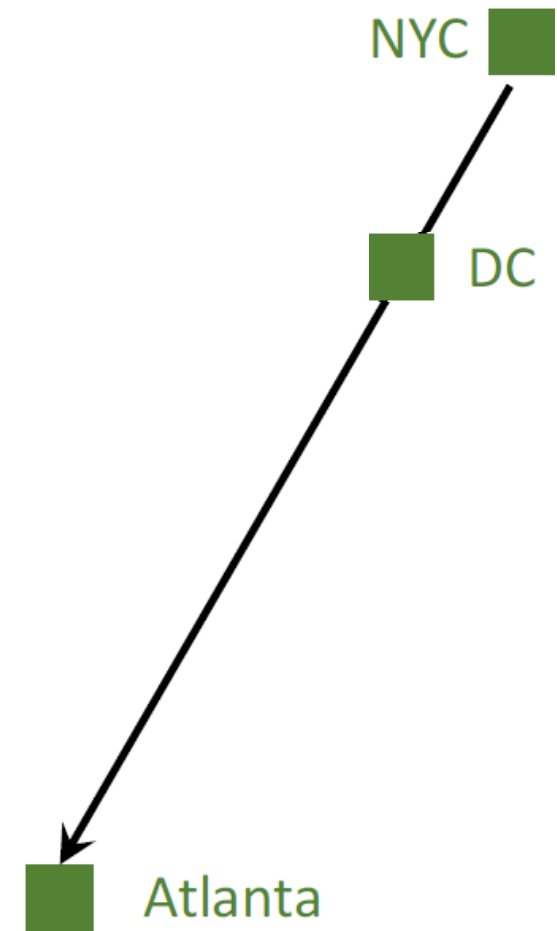
5.2、Temporal Difference (TD) Learning

➤ 举例

- 我想从纽约开车到亚特兰大。(通过华盛顿)
- 模型 $Q(w)$ 进行时间成本的估计, 例如1000min。

问题：如何更新模型？

- 可以在**结束旅行前** (获得 y 前) 更新模型吗？
- 一到达华盛顿我便能得到更好的 w 吗？



5.2、Temporal Difference (TD) Learning

- 模型的估计:

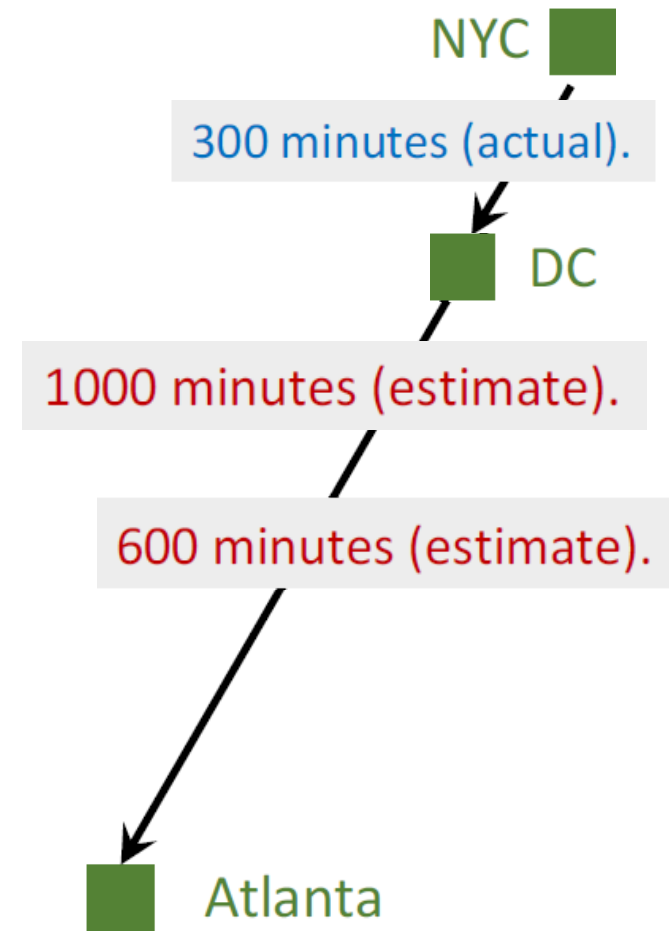
纽约到亚特兰大: 1000min (估计)。

- 到达华盛顿; 实际使用时间:

纽约到华盛顿: 300min (实际)。

- 模型现在更新了其估计:

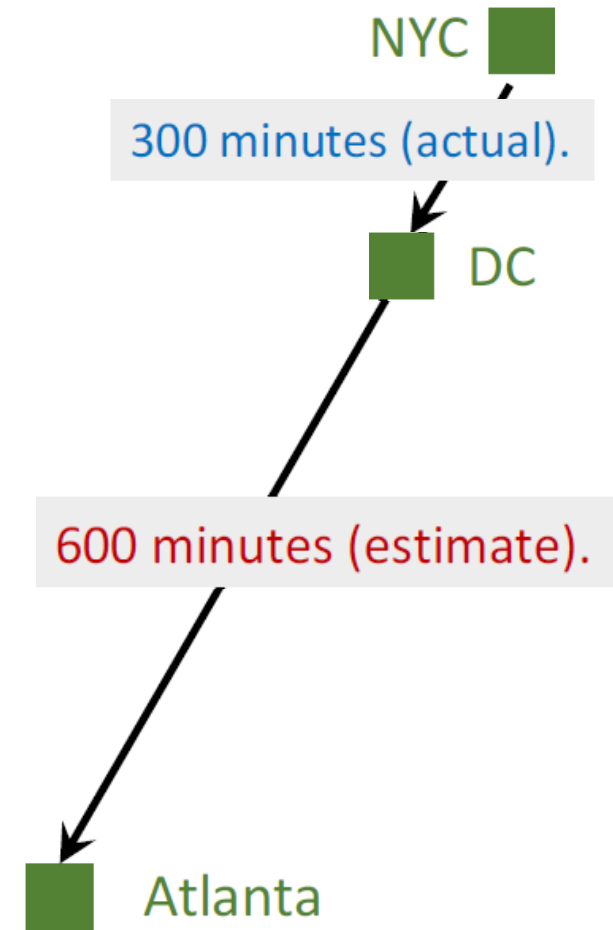
华盛顿到亚特兰大: 600min (估计)。



5.2、Temporal Difference (TD) Learning

- 模型的估计: $Q(w) = 1000 \text{ min}$
- 更新后的估计: $300 + 600 = 900 \text{ min}$

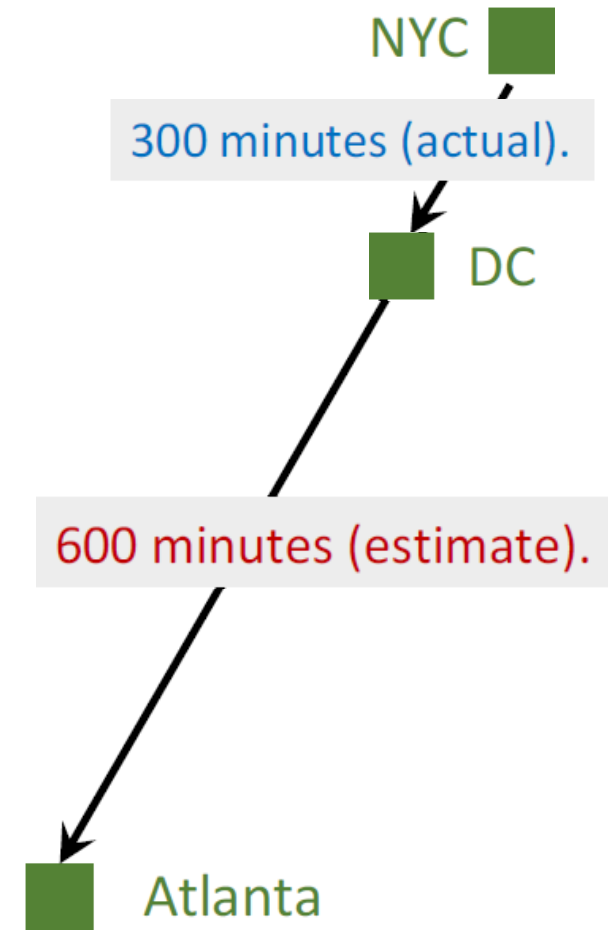
TD target
- TD 目标 $y = 900$ 是比 1000 更可靠的估计值。
- Loss: $L = \frac{1}{2} (Q(w) - y)^2$
- 梯度: $\frac{\partial L}{\partial w} = (\underbrace{1000 - 900}_{\text{TD error}}) \cdot \frac{\partial Q(w)}{\partial w}$
- 梯度下降: $w_{t+1} = w_t - \alpha \cdot \frac{\partial L}{\partial w} \big|_{w=w_t}$



5.2、Temporal Difference (TD) Learning

- 模型的估计: $Q(w) = 1000 \text{ min}$
- 更新后的估计: $300 + 600 = 900 \text{ min}$

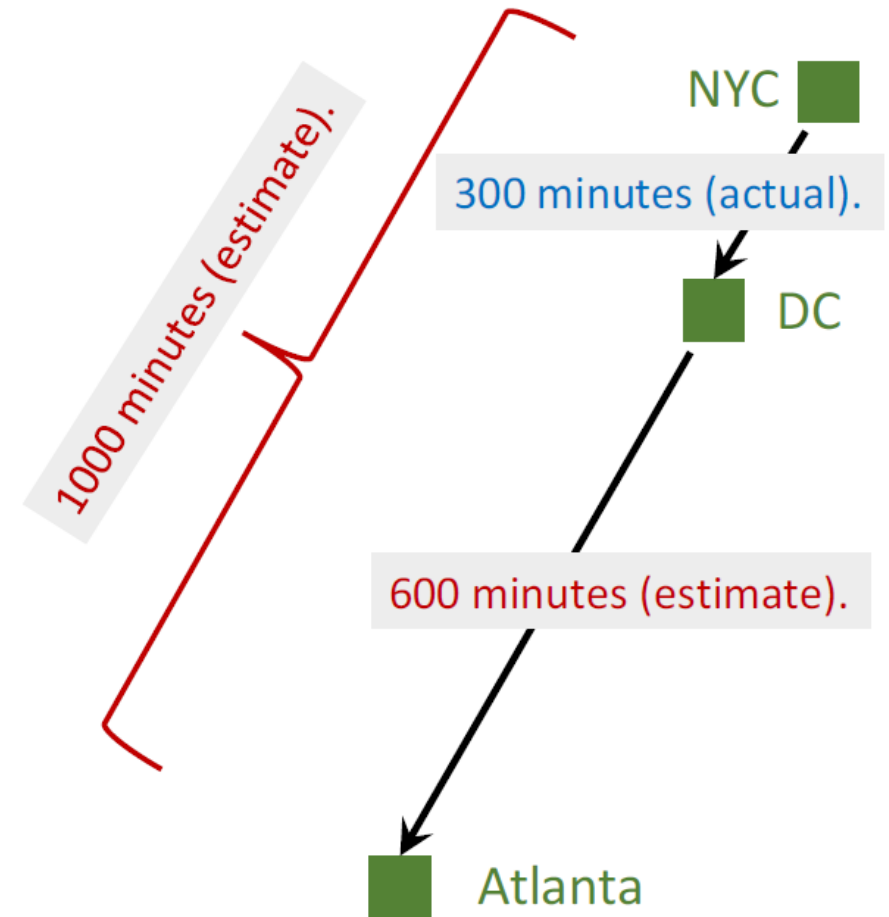
TD target
- TD 目标 $y = 900$ 是比 1000 更可靠的估计值。
- Loss: $L = \frac{1}{2}(Q(w) - y)^2$
- 梯度: $\frac{\partial L}{\partial w} = (1000 - 900) \cdot \frac{\partial Q(w)}{\partial w}$
- 梯度下降: $w_{t+1} = w_t - \alpha \cdot \frac{\partial L}{\partial w} |_{w=w_t}$



5.2、Temporal Difference (TD) Learning

➤为什么 TD Learning 有用?

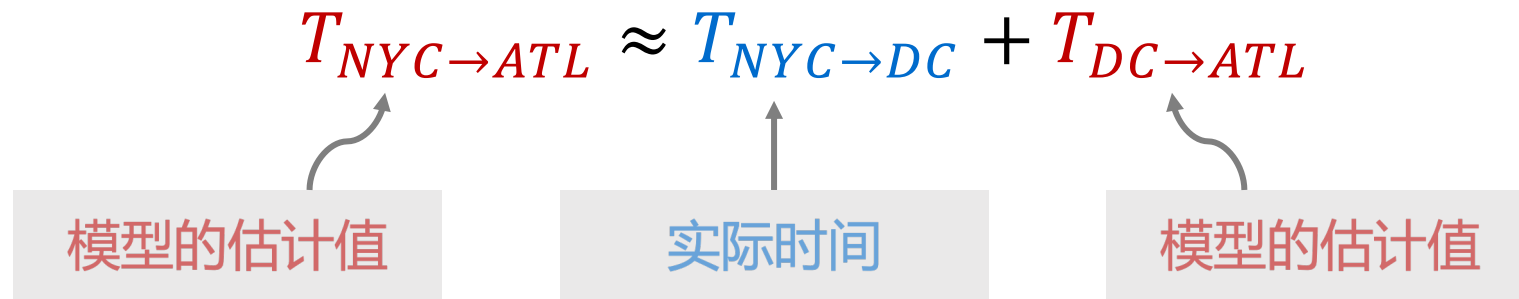
- 模型的估计:
 - NYC 到 Atlanta: 1000min
 - DC 到 Atlanta: 600min
 - NYC 到 DC: 400min
- 基本事实:
 - NYC 到 DC: 300min
- TD error: $\delta = 400 - 300 = 100$



5.3、TD Learning → DQN

➤ 如何将TD Learning 应用至DQN?

- 在“行驶时间”举例中，可得：



- 在深度强化学习中

$$Q(s_t, a_t; \mathbf{w}) \approx r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w})$$

5.3、TD Learning → DQN



- $U_t = R_t + \gamma \cdot U_{t+1}$

- $$\begin{aligned} U_t &= R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \gamma^4 \cdot R_{t+4} + \dots \\ &= R_t + \gamma \cdot (R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots) \\ &= R_t + \gamma \cdot (R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots) \\ &\qquad\qquad\qquad = U_{t+1} \end{aligned}$$

5.3、TD Learning → DQN



- $U_t = R_t + \gamma \cdot U_{t+1}$

TD Learning for DQN:

- DQN的输出, $Q(s_t, a_t; w)$, 是 U_t 一个估计值。
- DQN的输出, $Q(s_{t+1}, a_{t+1}; w)$, 是 U_{t+1} 一个估计值。
- 因此,
$$\underbrace{Q(s_t, a_t; w)}_{U_t \text{ 的估计值}} \approx E[R_t + \gamma \cdot \underbrace{Q(s_{t+1}, a_{t+1}; w)}_{U_{t+1} \text{ 的估计值}}]$$

5.3、TD Learning → DQN



- $U_t = R_t + \gamma \cdot U_{t+1}$

TD Learning for DQN:

- DQN的输出, $Q(s_t, a_t; w)$, 是 U_t 一个估计值。
- DQN的输出, $Q(s_{t+1}, a_{t+1}; w)$, 是 U_{t+1} 一个估计值。
- 因此, $Q(s_t, a_t; w) \approx r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; w)$

预测值

TD 目标值

5.3、TD Learning → DQN

➤ 使用TD Learning 训练DQN

- 预测值: $Q(s_t, a_t; w_t)$ 。
- TD目标值:

$$\begin{aligned} y_t &= r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; w_t) \\ &= r_t + \gamma \cdot \max_a Q(s_{t+1}, a; w_t) \end{aligned}$$

- Loss: $L_t = \frac{1}{2} [Q(s_t, a_t; w_t) - y_t]^2$
- 梯度下降: $w_{t+1} = w_t - \alpha \cdot \frac{\partial L_t}{\partial w} \big|_{w=w_t}$

5.2、Temporal Difference (TD) Learning

算法：TD学习的一次迭代

1. 观测状态 $S_t = s_t$ 和最合理的动作 $A_t = a_t$.
2. 预测值: $q_t = Q(s_t, a_t; w_t)$.
3. 区分价值网络: $d_t = \left. \frac{\partial Q(s_t, a_t; w)}{\partial w} \right|_{w=w_t}$
4. 环境提供新状态 s_{t+1} 和奖励 r_t .
5. 计算TD目标: $y_t = r_t + \gamma \times \max_a Q(s_{t+1}, a; w_t)$.
6. 梯度下降: $w_{t+1} = w_t - \alpha \times (q_t - y_t) \times d_t$

6.1、策略函数近似

➤ 策略函数 $\pi(a|s)$

- 策略函数 $\pi(a|s)$ 是一个概率密度函数。
- 该函数输入为 s
- 其输出所有行动的可能性，例如

$$\pi(\text{left} | s) = 0.2,$$

$$\pi(\text{right} | s) = 0.1,$$

$$\pi(\text{up} | s) = 0.7.$$

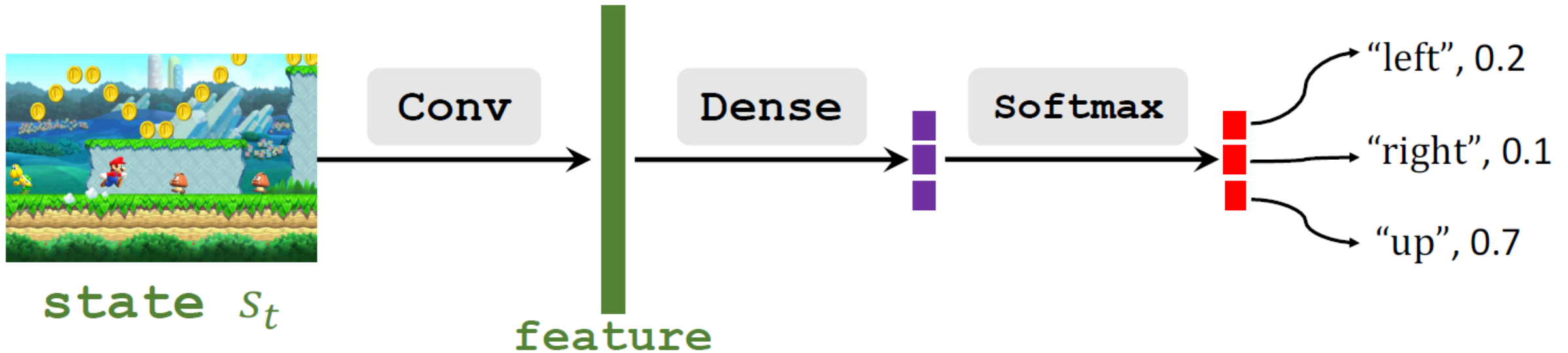
- 从分布中随机抽样行动 a

6.1、策略函数近似

➤ Policy Network $\pi(a|s; \theta)$

Policy Network: 使用神经网络近似 $\pi(a|s)$

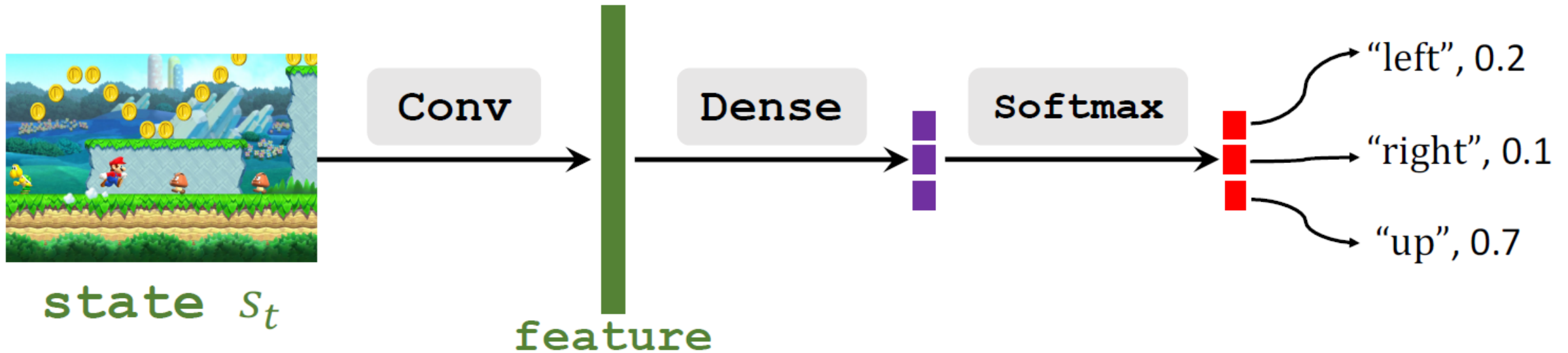
- 使用Policy Network $\pi(a|s; \theta)$ 近似 $\pi(a|s)$ 。
- θ : 神经网络的可训练参数



6.1、策略函数近似

➤ Policy Network $\pi(a|s; \theta)$

- $\sum_{a \in A} \pi(a|s; \theta) = 1$
- $A = \{ "left", "right", "up" \}$, 是所有行动的集合
- 因此使用softmax激活函数



6.2、状态价值函数近似



➤ 行动价值函数

定义： 折扣回报

- $U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$



- 回报依赖于状态 $s_t, s_{t+1}, \dots, s_n$ 和行动 $A_t, A_{t+1}, \dots, A_n$
- 行动时随机的: $P[A = a | S = s] = \pi(a | s)$ (策略函数)
- 状态是随机的: $P[S' = s' | S = s, A = a] = p(s' | s, a)$ (状态转移)

6.2、状态价值函数近似

➤ 行动价值函数

定义： 折扣回报

- $$U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$$

定义： 行动价值函数

- $$Q_\pi(s_t, a_t) = E[U_t | S_t = s_t, A_t = a_t]$$

定义： 状态价值函数

- $$V_\pi(s_t) = E_A [Q_\pi(s_t, A)] = \sum_a \pi(a | s_t) \cdot Q_\pi(s_t, a)$$

6.2、状态价值函数近似

➤ 基于策略的强化学习

定义：状态价值函数

- $V_{\pi}(s_t) = E_A [Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a)$

逼近状态价值函数

- 通过Policy Network $\pi(a|s_t; \theta)$ 逼近策略函数 $\pi(a|s_t)$
- 通过下列逼近价值函数

$$V_{\pi}(s_t; \theta) \approx \sum_a \pi(a|s_t; \theta) \cdot Q_{\pi}(s_t, a)$$

6.2、状态价值函数近似

➤ 基于策略的强化学习

逼近状态价值函数

- $V_{\pi}(s_t; \theta) = \sum_a \pi(a|s_t; \theta) \cdot Q_{\pi}(s_t, a)$

基于策略学习：学习 θ 最大化 $J(\theta) = E_s [V(s; \theta)]$

如何提高 θ ? Policy Gradient上升!

- 观察状态 s

- 更新策略: $\theta \leftarrow \theta + \beta \cdot \frac{\partial V(s; \theta)}{\partial \theta}$

Policy Gradient

6.3、Policy Gradient



定义：逼近状态价值函数

- $V(s; \theta) = \sum_a \pi(a|s; \theta) \cdot Q_\pi(s, a)$

Policy Gradient: $V(s; \theta)$ 关于 θ 的导数

- $$\begin{aligned} \frac{\partial V(s; \theta)}{\partial \theta} &= \frac{\partial \sum_a \pi(a|s; \theta) \cdot Q_\pi(s, a)}{\partial \theta} \\ &= \sum_a \frac{\partial \pi(a|s; \theta) \cdot Q_\pi(s, a)}{\partial \theta} \\ &= \sum_a \frac{\partial \pi(a|s; \theta)}{\partial \theta} \cdot Q_\pi(s, a) \end{aligned}$$

注：这种推导过于简化，不够严谨

6.3、Policy Gradient



定义：逼近状态价值函数

- $V(s; \theta) = \sum_a \pi(a|s; \theta) \cdot Q_\pi(s, a)$

Policy Gradient: $V(s; \theta)$ 关于 θ 的导数

- $$\begin{aligned} \frac{\partial V(s; \theta)}{\partial \theta} &= \sum_a \frac{\partial \pi(a|s; \theta)}{\partial \theta} \cdot Q_\pi(s, a) \\ &= \sum_a \pi(a|s; \theta) \boxed{\frac{\partial \log \pi(a|s; \theta)}{\partial \theta}} \cdot Q_\pi(s, a) \end{aligned}$$

- $$\frac{\partial \log [\pi(\theta)]}{\partial \theta} = \frac{1}{\pi(\theta)} \cdot \frac{\partial \pi(\theta)}{\partial \theta}$$

- $$\rightarrow \pi(\theta) \cdot \frac{\partial \log [\pi(\theta)]}{\partial \theta} = \cancel{\pi(\theta)} \cdot \frac{1}{\cancel{\pi(\theta)}} \cdot \frac{\partial \pi(\theta)}{\partial \theta}$$

6.3、Policy Gradient



定义：逼近状态价值函数

- $V(s; \theta) = \sum_a \pi(a|s; \theta) \cdot Q_\pi(s, a)$

Policy Gradient: $V(s; \theta)$ 关于 θ 的导数

- $$\begin{aligned} \frac{\partial V(s; \theta)}{\partial \theta} &= \sum_a \frac{\partial \pi(a|s; \theta)}{\partial \theta} \cdot Q_\pi(s, a) \\ &= \sum_a \pi(a|s; \theta) \frac{\partial \log \pi(a|s; \theta)}{\partial \theta} \cdot Q_\pi(s, a) \\ &= E_A \left[\frac{\partial \log \pi(A|s; \theta)}{\partial \theta} \cdot Q_\pi(s, A) \right] \end{aligned}$$

$A \sim \pi(\cdot | s; \theta)$

6.3、Policy Gradient



Policy Gradient

$$\frac{\partial V(s; \theta)}{\partial \theta} = \mathbb{E}_{A \sim \pi(\cdot | s; \theta)} \left[\frac{\partial \log \pi(A | s; \theta)}{\partial \theta} \cdot Q_{\pi}(s, A) \right]$$

6.3、Policy Gradient



Policy Gradient: $\frac{\partial V(s; \theta)}{\partial \theta} = \mathbb{E}_{A \sim \pi(\cdot | s; \theta)} \left[\frac{\partial \log \pi(A | s; \theta)}{\partial \theta} \cdot Q_{\pi}(s, A) \right]$

➤ 蒙特卡洛近似

- 根据 $\pi(\cdot | s; \theta)$ 随机采样一个行动 $\hat{a}$
 - 计算 $g(\hat{a}, \theta) = \frac{\partial \log \pi(\hat{a} | s; \theta)}{\partial \theta} \cdot Q_{\pi}(s, \hat{a})$
 - 使用 $g(\hat{a}, \theta)$ 作为 Policy Gradient $\frac{\partial V(s; \theta)}{\partial \theta}$ 的近似值
- $\mathbb{E}_A[g(A, \theta)] = \frac{\partial V(s; \theta)}{\partial \theta}$
 - $g(\hat{a}, \theta)$ 是 $\frac{\partial V(s; \theta)}{\partial \theta}$ 的无偏估计

6.4、使用Policy Gradient更新Policy Network

➤ 算法

1. 观察状态 s_t
2. 根据 $\pi(\cdot | s_t; \theta_t)$ 随机采样一个行动 a_t
3. 计算 $q_t \approx Q_\pi(s_t, a_t)$ (一些估计值)
4. Policy Network微分: $\mathbf{d}_{\theta,t} \frac{\partial \log \pi(a_t | s_t, \theta)}{\partial \theta} \big|_{\theta=\theta_t}$
5. (近似) Policy Gradient: $\mathbf{g}(a_t, \theta_t) = q_t \cdot \mathbf{d}_{\theta,t}$
6. 更新Policy Network: $\theta_{t+1} = \theta_t + \beta \cdot \mathbf{g}(a_t, \theta_t)$

6.4、使用Policy Gradient更新Policy Network

➤ 算法

1. 观察状态 s_t
2. 根据 $\pi(\cdot | s_t; \theta_t)$ 随机采样一个行动 a_t
3. 计算 $q_t \approx Q_\pi(s_t, a_t)$ (一些估计值) **How?**
4. Policy Network微分: $\mathbf{d}_{\theta,t} \frac{\partial \log \pi(a_t | s_t, \theta)}{\partial \theta} \big|_{\theta=\theta_t}$
5. (近似) Policy Gradient: $\mathbf{g}(a_t, \theta_t) = q_t \cdot \mathbf{d}_{\theta,t}$
6. 更新Policy Network: $\theta_{t+1} = \theta_t + \beta \cdot \mathbf{g}(a_t, \theta_t)$

6.4、使用Policy Gradient更新Policy Network

➤ 算法

计算 $q_t \approx Q_\pi(s_t, a_t)$ (一些估计值)

How?

方法1: Reinforce

- 玩完游戏得到trajectory

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T$$

- 计算所有 t 折扣回报 $u_t = \sum_{k=t}^T \gamma^{k-t} r_k$
- 因为 $Q_\pi(s_t, a_t) = E[U_t]$, 因此可以使用 u_t 近似 $Q_\pi(s_t, a_t)$
- $\rightarrow$ 使用 $q_t = u_t$

6.4、使用Policy Gradient更新Policy Network

➤ 算法

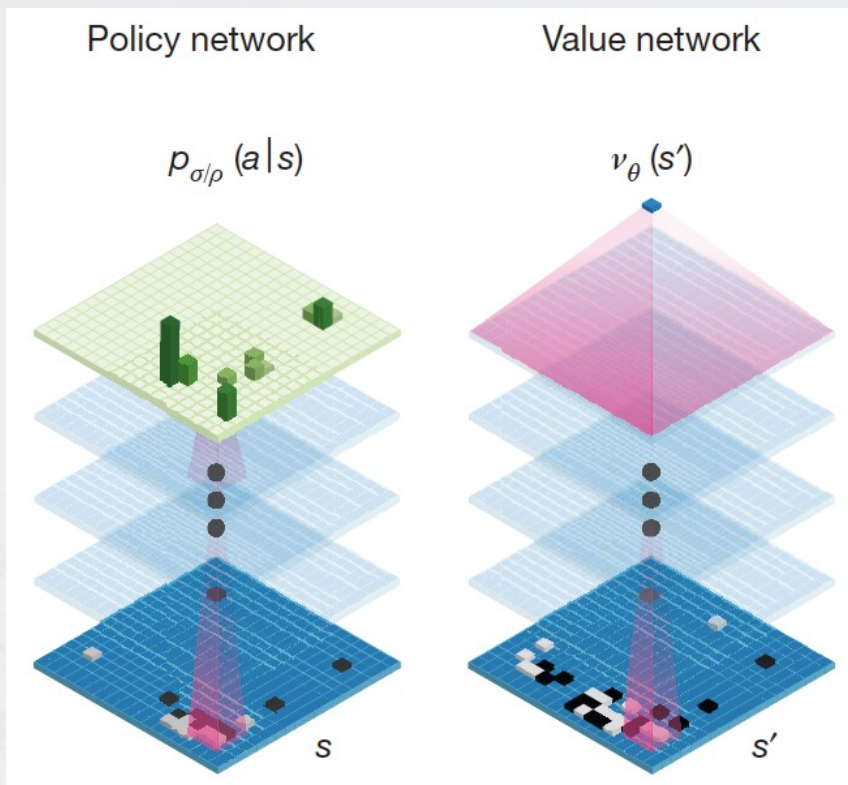
计算 $q_t \approx Q_\pi(s_t, a_t)$ (一些估计值)

How?

方法2：使用神经网络近似 Q_π

- Actor-Critic method

• 应用—AlphaGo



- 策略网络: 以人类棋谱进行监督学习初始化, 然后策略梯度方法改善
- 值网络: 基于自我对弈训练输出的预测值
- 网络输出用于 Monte Carlo tree search (MCTS)

D. Silver et al., [Mastering the Game of Go with Deep Neural Networks and Tree Search](#),
Nature 529, January 2016

• 本章小结

□ 强化学习基本术语

□ 建立强化学习的基本任务：MDP四元组

□ 多摇臂赌博机是单步强化学习的范例： ϵ 贪心算法和softmax算法

□ 基于价值的强化学习方法：DQN, TD gradient

□ 基于策略的强化学习方法